

CODESCAPE - JOC 3D EDUCATIV PENTRU INVĂȚAREA CONCEPTELOR DE PROGRAMARE

Candidat: Adelina-Simina Jurca

Coordonator științific: Valer Bocan

Sesiunea: Iulie 2026

REZUMAT

Lucrarea de față prezintă proiectarea și dezvoltarea aplicației CodeScape, un joc educativ 3D realizat în Unity, conceput pentru a sprijini înțelegerea unor concepte introductive de programare printr-o experiență interactivă. Ideea proiectului a pornit de la observația că multe noțiuni studiate la începutul formării în informatică sunt percepute ca fiind abstracte și greu de urmărit atunci când sunt prezentate exclusiv teoretic.

Prin urmare, aplicația propusă încearcă să transforme noțiuni precum reprezentarea binară, logica booleană, structura de date stivă, instrucțiunile condiționale și structurile repetitive în activități concrete, ușor de urmărit de către utilizator. Fiecare nivel are o temă clară și o regulă de validare proprie, iar progresul este construit astfel încât utilizatorul să parcurgă jocul într-o ordine firească și să primească feedback imediat.

Din punct de vedere tehnic, proiectul integrează un mediu 3D first-person, obiecte interactive, mesaje contextuale, verificări de stare și componente audio-vizuale menite să susțină procesul de învățare. Logica principală a aplicației este implementată în C#, iar organizarea conținutului s-a realizat prin scene, prefabs și manageri dedicați fiecărui tip de comportament.

Lucrarea urmărește atât descrierea soluției propuse, cât și explicarea etapelor de proiectare, implementare și testare. Rezultatul final este o aplicație cu rol educativ care demonstrează că mediile interactive pot fi folosite eficient pentru introducerea unor concepte de bază din programare și că o abordare vizuală poate crește gradul de implicare și claritate pentru utilizatorii aflați la început.

ABSTRACT

This paper describes CodeScape, a 3D educational game built in Unity that helps beginners explore basic programming concepts through interactive tasks. The project started from the observation that many beginner-level programming topics are often perceived as abstract and difficult to follow when they are taught only in a theoretical manner.

The proposed application transforms ideas such as binary representation, boolean logic, stack-based ordering, conditional statements and repetitive structures into concrete game tasks. Each level focuses on one concept and asks the player to solve a task before moving on, so the game advances step by step in a beginner-friendly way.

From a technical perspective, the application combines a first-person 3D environment, interactive objects, contextual text panels, state validation and audio-visual feedback meant to support the learning process. The main logic is implemented in C#, while the content is organized through scenes, prefabs and dedicated manager scripts.

The paper explains the proposed solution, the design decisions, the implementation process and the testing stage. The project shows that a 3D game environment can make basic programming concepts easier to approach by connecting them with visible actions and immediate feedback.

CUPRINS

CAPITOLUL 1. INTRODUCERE	6
1.1 Contextul general al problemei	6
1.2 Tema proiectului	6
1.3 Motivația alegerii temei	7
1.4 Obiectivele lucrării	7
1.5 Contribuții personale	8
1.6 Structura lucrării.....	8
CAPITOLUL 2. ANALIZA STADIULUI ACTUAL ÎN DOMENIUL PROBLEMEI	9
2.1 Rolul aplicațiilor interactive în învățarea programării	9
2.2 Scratch	9
2.3 Lightbot.....	9
2.4 CodeCombat	10
2.5 Human Resource Machine	10
2.6 Concluzii comparative.....	11
CAPITOLUL 3. BAZELE TEORETICE ȘI TEHNOLOGIILE UTILIZATE	12
3.1 Motorul Unity	12
3.2 Limbajul C#.....	12
3.3 TextMeshPro și interfața cu utilizatorul	12
3.4 Scene, prefabs și organizarea conținutului	13
3.5 Fizică, coliziuni și input	13
3.6 Conceptele de programare integrate în joc.....	14
3.6.1 Reprezentarea binară	14
3.6.2 Logica booleană.....	14
3.6.3 Structura de date stivă	14
3.6.4 Instrucțiunile condiționale.....	14
3.6.5 Structurile repetitive	15
CAPITOLUL 4. SOLUȚIA PROPUȘĂ ȘI METODOLOGIA DE DEZVOLTARE...	16
4.1 Cerințe funcționale.....	16
4.2 Cerințe nefuncționale.....	16
4.3 Scenariul general de utilizare	16
4.4 Fluxul aplicației	17
4.5 Camera de spawn	17
4.6 Proiectarea nivelului 1 – conversie binar-zecimal.....	19
4.7 Proiectarea nivelului 2 – logică booleană	21
4.8 Proiectarea nivelului 3 – structură de tip stivă	23
4.9 Proiectarea nivelului 4 – instrucțiuni condiționale	25
4.10 Proiectarea nivelului 5 – structuri repetitive	27

4.11 Elemente de interfață și feedback	28
4.12 Metodologia de dezvoltare	30
CAPITOLUL 5. IMPLEMENTAREA APLICAȚIEI CODESCAPE	31
5.1 Organizarea proiectului și structura generală	31
5.2 Implementarea controlului jucătorului	31
5.3 Gestionarea nivelurilor și a progresului	32
5.4 Terminale informative și panoul de informații	34
5.5 Implementarea nivelului 1 – reprezentarea binară.....	36
5.6 Implementarea nivelului 2 – logică booleană.....	38
5.7 Implementarea nivelului 3 – structură de tip stivă.....	39
5.8 Implementarea nivelului 4 – decizii condiționale.....	41
5.9 Implementarea nivelului 5 – structuri repetitive și contorizare	42
5.10 Finalizarea jocului și ecranul final	44
CAPITOLUL 6. TESTAREA APLICAȚIEI	46
6.1 Strategia de testare	46
6.2 Teste funcționale	46
6.3 Probleme întâmpinate și remedieri	47
6.4 Observații privind experiența de utilizare	48
6.5 Limite actuale ale aplicației.....	48
CAPITOLUL 7. CONCLUZII ȘI DEZVOLTĂRI VIITOARE	49
7.1 Concluzii generale	49
7.2 Gradul de atingere a obiectivelor	49
7.3 Dezvoltări viitoare	49
BIBLIOGRAFIE	51

CAPITOLUL 1. INTRODUCERE

1.1 Contextul general al problemei

În ultimii ani, aplicațiile educaționale interactive au devenit tot mai prezente în procesul de învățare, în special în domeniile în care există un volum mare de noțiuni abstracte. Programarea face parte din această categorie, deoarece presupune înțelegerea unor reguli, structuri și mecanisme care nu sunt întotdeauna ușor de vizualizat de către un începător. În multe situații, studentul sau elevul poate memora definiții, dar întâmpină dificultăți atunci când trebuie să înțeleagă de ce o anumită logică produce un anumit rezultat.

Din acest motiv, trecerea de la explicația teoretică la o experiență practică este esențială. O aplicație interactivă poate să creeze un context mai clar pentru noțiuni precum numărul de iterații, valoarea unei expresii logice sau ordinea în care sunt manipulate datele. Faptul că utilizatorul vede imediat urmarea unei alegeri îl ajută să înțeleagă mai bine regula aplicată.

Pentru învățarea programării sunt disponibile mai multe platforme și jocuri educaționale care folosesc activități vizuale, blocuri de comandă sau situații de joc pentru a explica noțiuni de bază. Totuși, multe dintre ele fie mizează pe o interfață bidimensională, fie insistă din timp pe sintaxa propriu-zisă a unui limbaj. CodeScape a fost gândit ca o variantă mai apropiată de explorarea spațială și de interacțiunea directă cu obiecte dintr-un mediu 3D.

1.2 Tema proiectului

Tema acestei lucrări o reprezintă proiectarea și dezvoltarea aplicației CodeScape, un joc educativ 3D conceput pentru învățarea unor concepte de bază din programare prin interacțiune directă cu un mediu virtual. Aplicația a fost realizată în Unity și este organizată sub forma unei experiențe secvențiale, în care utilizatorul parcurge mai multe niveluri și rezolvă puzzle-uri asociate unor noțiuni studiate în mod obișnuit la începutul formării în informatică.

În locul unei prezentări exclusiv teoretice, proiectul propune o abordare mai practică și mai accesibilă. Jucătorul nu primește doar informații, ci este pus în situația de a observa, testa, greși și corecta, iar acest mod de interacțiune poate face procesul de învățare mai clar și mai ușor de urmărit. Din această perspectivă, CodeScape îmbină rolul de aplicație software cu cel de instrument educațional.

Structura jocului are și un avantaj didactic: fiecare nivel este suficient de concentrat pe o singură idee pentru a evita încărcarea inutilă a utilizatorului, dar în același timp toate nivelurile sunt legate între ele printr-un fir comun de progres. Astfel, experiența nu este o colecție de mini-exerciții separate, ci o aplicație coerentă.

1.3 Motivația alegerii temei

Alegerea acestei teme a fost influențată atât de interesul meu pentru dezvoltarea de aplicații interactive, cât și de experiența personală acumulată pe parcursul studiilor. Fiind studentă în anul al III-lea la Facultatea de Automatică și Calculatoare, specializarea Informatică, m-am confruntat de mai multe ori cu dificultatea de a înțelege practic anumite concepte introductive din programare. Explicațiile teoretice sunt importante, însă pentru un începător ele devin mai clare atunci când sunt completate de exemple practice și situații ușor de urmărit.

Pornind de la această observație, am considerat utilă realizarea unei aplicații care să transforme conceptele de bază din programare în activități interactive, mai ușor de urmărit și de înțeles. Mi s-a părut important ca utilizatorul să nu rămână doar la nivelul definițiilor, ci să poată vedea și aplica în mod direct ideile prezentate. Din acest motiv, am ales să dezvolt un joc educativ 3D în care noțiuni precum reprezentarea binară, logica booleană, structura de date stivă, instrucțiunile condiționale și structurile repetitive să fie integrate în puzzle-uri și situații practice.

Consider că o astfel de aplicație nu este utilă doar din perspectiva propriului proces de învățare, ci și pentru alți elevi sau studenți care se confruntă cu aceeași problemă. În același timp, tema mi-a oferit posibilitatea de a îmbina două direcții care mă interesează în mod deosebit: dezvoltarea de jocuri și realizarea unei aplicații cu rol educațional. Astfel, proiectul nu a fost gândit doar ca un exercițiu tehnic realizat în Unity, ci ca o soluție practică prin care învățarea unor concepte introductive din programare poate deveni mai clară, mai accesibilă și mai atractivă.

1.4 Obiectivele lucrării

Prin realizarea aplicației CodeScape s-au urmărit mai multe obiective, atât din punct de vedere tehnic, cât și din punct de vedere educațional:

- crearea unui mediu 3D interactiv, ușor de parcurs și suficient de clar pentru un utilizator aflat la început;
- implementarea unor niveluri tematice dedicate unor concepte fundamentale de programare;
- realizarea unui sistem de progres care să încurajeze rezolvarea etapizată a provocărilor;
- oferirea unui feedback imediat prin mesaje, efecte vizuale și elemente audio;
- dezvoltarea unei aplicații coerente, funcționale și ușor de extins pe viitor.

Dincolo de aceste obiective generale, proiectul și-a propus și un scop mai pragmatic: fiecare nivel să poată fi explicat ușor atât din perspectiva utilizatorului, cât și din perspectiva codului. Această dublă lizibilitate este utilă și în redactarea lucrării, pentru că permite legătura directă dintre ceea ce se vede în aplicație și ceea ce există în implementare.

1.5 Contribuții personale

Contribuția personală în cadrul proiectului se reflectă atât în definirea conceptului general, cât și în implementarea practică a nivelurilor, a mecanismilor de interacțiune și a structurii de progres. Aplicația nu a fost gândită doar ca o demonstrație tehnică, ci ca un produs cu o logică internă clară și cu o legătură directă între conținutul didactic și forma de prezentare.

De asemenea, o contribuție importantă constă în transpunerea fiecărui concept de programare într-un puzzle distinct, suficient de simplu pentru a fi intuitiv, dar suficient de clar pentru a reflecta ideea teoretică de bază. Alegerea acestor mecanici și adaptarea lor într-un spațiu 3D reprezintă una dintre părțile definitorii ale proiectului.

1.6 Structura lucrării

Lucrarea este organizată în șapte capitole. Primul capitol introduce contextul general, tema proiectului, motivația alegerii acesteia și obiectivele urmărite. Capitolul al doilea este dedicat analizei unor aplicații sau abordări similare din zona jocurilor educaționale și a instrumentelor de învățare a programării.

Capitolul al treilea prezintă bazele teoretice și tehnologiile utilizate în realizarea proiectului. Capitolul al patrulea descrie soluția propusă și metodologia de dezvoltare, iar capitolul al cincilea detaliază implementarea principalelor funcționalități. Capitolul al șaselea prezintă testarea aplicației, iar la final sunt formulate concluziile și câteva direcții pentru dezvoltări viitoare.

CAPITOLUL 2. ANALIZA STADIULUI ACTUAL ÎN DOMENIUL PROBLEMEI

2.1 Rolul aplicațiilor interactive în învățarea programării

În cazul începătorilor, dificultatea principală nu este întotdeauna memorarea unor definiții, ci înțelegerea relației dintre o acțiune și rezultatul produs. Din această cauză, aplicațiile interactive pot avea un rol important în învățarea programării, deoarece oferă feedback imediat și transformă o noțiune abstractă într-o situație concretă.

Un alt avantaj al acestui tip de aplicații este faptul că pot reduce teama de greșeală. Într-un joc sau într-un mediu experimental, utilizatorul este încurajat să încerce variante diferite, să observe consecințe și să corecteze singur anumite decizii. În cazul programării, acest lucru este foarte util, pentru că o parte importantă din procesul real de lucru presupune testare și corectare.

Există însă diferențe mari între tipurile de aplicații existente. Unele mizează pe simplificarea maximă și folosesc blocuri vizuale, altele introduc mai devreme scrierea de cod, iar altele tratează învățarea prin puzzle-uri care pun accentul pe secvențialitate și logică. În continuare sunt analizate câteva exemple cunoscute, relevante pentru poziționarea aplicației CodeScape.

2.2 Scratch

Scratch este una dintre cele mai cunoscute platforme de introducere în programare, mai ales pentru elevi și utilizatori foarte tineri. Elementul central al platformei îl reprezintă programarea prin blocuri vizuale, ceea ce reduce mult dificultatea produsă de sintaxă și permite concentrare pe logică și secvențialitate [1].

Avantajul major al acestei abordări este accesibilitatea. Utilizatorul poate să înțeleagă ușor relația dintre instrucțiuni și comportamentul rezultat, iar feedback-ul este rapid. În plus, mediul încurajează experimentarea și recompensează creativitatea, deoarece proiectele pot fi modificate și extinse fără costuri mari de înțelegere.

Totuși, Scratch rămâne predominant bidimensional și lucrează cu o logică de construcție diferită de cea a unui joc 3D explorabil. Din această perspectivă, CodeScape se diferențiază prin accentul pus pe mișcare într-un spațiu tridimensional și pe asocierea fiecărui concept cu o sarcină fizic reprezentată în joc.

- punct forte: foarte accesibil pentru începători;
- limită: experiența nu pune accent pe explorare 3D sau pe imersiune spațială.

2.3 Lightbot

Lightbot este un joc educațional bazat pe puzzle-uri, în care utilizatorul controlează un personaj prin secvențe de comenzi. Din punct de vedere didactic, aplicația este utilă

pentru introducerea unor idei precum secvențierea, procedurile și buclele, fără a solicita scrierea explicită de cod [2].

Meritul principal al jocului este claritatea regulilor. Fiecare nivel are un scop bine delimitat, iar utilizatorul înțelege rapid că ordinea instrucțiunilor influențează direct rezultatul. Acest tip de structură inspiră și organizarea unui nivel educațional eficient: o idee, o regulă și un feedback clar.

În același timp, Lightbot nu are un cadru de explorare liberă și nici un fir narativ spațial. CodeScape preia ideea de puzzle dedicat unei anumite noțiuni, dar o integrează într-o aplicație mai apropiată de parcursul unui joc 3D, ceea ce schimbă modul de implicare al utilizatorului.

- punct forte: puzzle-uri clare și ușor de înțeles;
- limită: mediul este mai rigid și nu oferă explorare tridimensională.

2.4 CodeCombat

CodeCombat este o platformă care introduce programarea într-un context de joc și folosește scrierea de cod ca principal mecanism de interacțiune. Utilizatorul observă consecința comenzilor sale direct în joc, ceea ce creează o legătură puternică între cod și comportament [3].

Comparativ cu platformele bazate pe blocuri, CodeCombat solicită mai devreme familiarizarea cu sintaxa, iar acest lucru poate fi un avantaj pentru utilizatorii care doresc trecerea rapidă către programarea propriu-zisă. În același timp, pentru un începător complet, această cerință poate reprezenta o barieră suplimentară.

CodeScape se poziționează într-o etapă anterioară: utilizatorul nu este obligat să scrie cod, ci să înțeleagă ideea din spatele lui. Din acest motiv, aplicația poate funcționa ca punte între explicația teoretică și un mediu ulterior în care ar fi necesară și sintaxa.

- punct forte: leagă direct codul de rezultatul vizibil în joc;
- limită: cere mai repede înțelegerea sintaxei decât o face CodeScape.

2.5 Human Resource Machine

Human Resource Machine este un joc cunoscut pentru modul în care transformă idei de algoritmică și manipulare de date în puzzle-uri cu reguli bine definite. Jucătorul rezolvă sarcini prin secvențe de instrucțiuni, iar accentul cade pe eficiență, corectitudine și înțelegerea logicii din spate [4].

Un aspect relevant pentru această lucrare este felul în care jocul reușește să păstreze tonul ludic fără să piardă componenta didactică. Puzzle-urile nu sunt doar decorative, ci chiar reflectă ideea pe care trebuie să o înțeleagă utilizatorul. Această legătură dintre formă și concept este importantă și în cazul CodeScape.

Diferența principală este că Human Resource Machine folosește o prezentare vizuală 2D și o abstractizare diferită a acțiunilor. CodeScape încearcă să obțină același tip de claritate conceptuală, dar prin interacțiune în spațiu 3D și prin obiecte care pot fi percepute mai direct de utilizator.

- punct forte: legătură puternică între puzzle și conceptul logic;
- limită: nu oferă un spațiu explorabil și nu urmărește aceeași imersiune.

2.6 Concluzii comparative

Analiza aplicațiilor de mai sus arată că toate încearcă, în forme diferite, să reducă distanța dintre o idee teoretică și o activitate practică. Unele fac acest lucru prin blocuri vizuale, altele prin puzzle-uri secvențiale sau prin introducerea codului într-un context de joc. CodeScape se plasează între aceste direcții, încercând să păstreze accesibilitatea și claritatea, dar și să adauge un nivel mai ridicat de imersiune.

Prin comparație, aplicația propusă nu urmărește să înlocuiască platformele consacrate, ci să ofere un cadru diferit pentru noțiuni introductive. Accentul pus pe explorarea 3D, pe progresul de la un nivel la altul și pe legătura dintre spațiul de joc și conceptul explicat reprezintă elementele care diferențiază proiectul de alte soluții analizate.

Tabelul 2.1. Comparație între aplicațiile analizate și CodeScape

Caracteristică	Scratch	Lightbot	CodeCombat	Human Resource Machine	CodeScape
Public țintă	începători și elevi	copii și începători	elevi și adolescenți	utilizatori obișnuiți cu puzzle-uri	începători interesați de o experiență 3D
Tipul interacțiunii	blocuri vizuale	puzzle logic	scriere de cod în context de joc	puzzle secvențial	explorare 3D și puzzle-uri tematice
Mediu 3D	nu	nu	parțial	nu	da
Concepte de programare	largi, introduse gradual	secvențe și bucle	mai multe concepte, inclusiv sintaxă	algoritmică și optimizare	binar, logică, stivă, if, bucle
Accent pe feedback imediat	ridicat	ridicat	ridicat	ridicat	ridicat
Necesitatea cunoașterii sintaxei	foarte redusă	nu	da	redușă	redușă
Accent pe imersiune	scăzut	scăzut	mediu	mediu	ridicat
Parcurgere secvențială	opțională	da	da	da	da

CAPITOLUL 3. BAZELE TEORETICE ȘI TEHNOLOGIILE UTILIZATE

3.1 Motorul Unity

Unity este mediul principal folosit pentru dezvoltarea aplicației CodeScape. Alegerea lui a fost justificată de flexibilitatea oferită în construirea unui mediu 3D, de suportul bun pentru organizarea pe scene și obiecte și de integrarea facilă a componentelor de fizică, sunet și interfață [5].

Pentru proiectul de față, Unity a fost util nu doar ca motor grafic, ci și ca spațiu de organizare a logicii. Fiecare nivel a putut fi tratat ca un ansamblu de obiecte, componente și reguli, iar sistemul de prefabs a permis reutilizarea unor elemente comune, precum camerele, ușile sau obiectele interactive [6].

Un alt motiv important al alegerii a fost faptul că Unity permite trecerea relativ ușoară de la idee la prototip. Într-un proiect educațional, acest aspect contează, deoarece dezvoltarea presupune multe ajustări de interacțiune și de prezentare până când un nivel devine suficient de clar pentru utilizator. Proiectul a fost realizat și testat în Unity Editor 2022.3.62f3, versiune LTS care a oferit stabilitate și compatibilitate bună cu pachetele utilizate.

3.2 Limbajul C#

Logica aplicației este implementată în C#, limbajul standard folosit în ecosistemul Unity. Din punct de vedere practic, C# a permis structurarea proiectului în clase cu responsabilități clare: controlul jucătorului, gestionarea panourilor informative, validarea nivelurilor și coordonarea progresului [7].

În acest proiect, C# este important nu doar pentru partea de implementare, ci și pentru felul în care susține o organizare clară a codului. Metodele și clasele sunt suficient de expresive pentru a descrie comportamente diferite, iar această claritate este utilă inclusiv atunci când codul trebuie explicat în cadrul lucrării.

3.3 TextMeshPro și interfața cu utilizatorul

Pentru afișarea textelor și a unor mesaje esențiale pentru utilizator a fost utilizat TextMeshPro. Acesta oferă o calitate bună a randării textului și este potrivit pentru mesaje de feedback, panouri informative și etichete vizibile în scenă [8].

Într-o aplicație educațională, textul are un rol mai mare decât într-un joc obișnuit. El trebuie să completeze acțiunea fără să o blocheze și să explice exact atât cât este necesar. Din acest motiv, soluția adoptată în proiect a fost bazată pe informații scurte, clare și ușor de parcurs.

Pentru textele principale din interfață a fost ales fontul Jura Variable [9], deoarece se potrivește vizual cu atmosfera science-fiction a aplicației și rămâne lizibil în panourile informative și în mesajele de stare.

În plus, textele de feedback și mesajele importante din interfață au fost evidențiate prin variații de culoare, text îngroșat și diferențe de dimensiune, astfel încât utilizatorul să poată distinge rapid informațiile esențiale de cele secundare.

3.4 Scene, prefabs și organizarea conținutului

O parte importantă din proiectare a constat în organizarea conținutului într-o formă ușor de administrat. Structura generală a jocului se bazează pe un prefab comun de tip cameră, care este instanțiat de fiecare dată de către LevelManager la încărcarea unei noi etape. Acest roomPrefab oferă baza arhitecturală reutilizabilă a jocului, incluzând spațiul general, elementele de trecere și suportul pentru afișarea informațiilor de nivel.

Pentru fiecare etapă, în interiorul acestei camere este încărcat un prefab dedicat nivelului respectiv. În acest fel, toate nivelurile păstrează aceeași structură generală, dar conținutul specific diferă în funcție de conceptul de programare prezentat. Această abordare reduce duplicarea și simplifică modificările ulterioare, deoarece separă componenta comună a mediului de componenta specifică fiecărui puzzle.

Din punct de vedere vizual, mediul 3D al aplicației este realizat în proporție foarte mare cu ajutorul pachetului gratuit „Sci-Fi Styled Modular Pack” [10]. Majoritatea elementelor vizibile din camere, precum podelele, pereții, luminile, consolele, containerele și alte obiecte decorative, provin din prefabs incluse în acest asset pack. Aceste resurse au oferit baza vizuală a aplicației și au contribuit la definirea atmosferei science-fiction urmărite în proiect.

Elementele specifice fiecărui nivel sunt apoi controlate de scriptul asociat celui puzzle. Astfel, fiecare manager de nivel, cum ar fi BitManager, ReactorPuzzleManager, StackPuzzleManager, IfPuzzleManager sau LoopPuzzleManager, se ocupă de logica proprie nivelului, de validarea condițiilor și de interacțiunea cu obiectele. În unele cazuri, scriptul doar gestionează obiecte deja existente în prefab-ul nivelului, iar în altele generează și elemente dinamice în timpul execuției, în funcție de cerințele puzzle-ului.

3.5 Fizică, coliziuni și input

Fiind o aplicație 3D cu perspectivă first-person, CodeScape folosește componente de fizică pentru mișcare, săritură și interacțiunea cu anumite obiecte, iar contactul jucătorului cu solul este verificat printr-un raycast orientat în jos. Aceste elemente sunt importante deoarece o bună parte din înțelegerea jocului se bazează pe relația dintre spațiu, obiect și acțiune. Sistemul de input a fost păstrat simplu, folosind tastele de deplasare, săritură și click pentru interacțiune.

Pe lângă mișcare și interacțiune, au fost introduse și câteva măsuri suplimentare de siguranță pentru a preveni comportamente nedorite ale obiectelor și ale jucătorului. Acestea includ limitarea unor poziționări incorecte, folosirea unor coliziuni de protecție și readucerea anumitor obiecte într-o stare controlată atunci când situația de joc o impune.

3.6 Conceptele de programare integrate în joc

Aplicația CodeScape a fost concepută astfel încât fiecare nivel să corespundă unui concept introductiv din programare. Ideea de bază a fost transformarea unor noțiuni teoretice în situații interactive, ușor de observat și de înțeles într-un mediu 3D. În locul unei prezentări abstracte, utilizatorul interacționează direct cu obiecte și primește feedback imediat în funcție de acțiunile realizate. În plus, fiecare nivel este însoțit de un ghid scurt, accesibil printr-un terminal informativ, care explică pe scurt conceptul teoretic necesar rezolvării puzzle-ului. Conceptele selectate pentru joc sunt reprezentarea binară, logica booleană, structura de date stivă, instrucțiunile condiționale și structurile repetitive.

3.6.1 Reprezentarea binară

Primul concept introdus în joc este reprezentarea binară. Din perspectivă teoretică, ideea este esențială pentru înțelegerea modului în care informația este reprezentată la nivel elementar. În aplicație, această noțiune este tradusă într-o succesiune de cuburi care pot avea valori 0 sau 1, iar utilizatorul trebuie să obțină combinația corectă.

3.6.2 Logica booleană

Al doilea concept important este logica booleană. Utilizatorul nu vede direct o expresie formală complicată, ci observă legătura dintre starea mai multor obiecte și rezultatul final. În acest fel, regulile AND, OR sau negarea devin mai ușor de înțeles prin asocierea cu un comportament vizibil.

3.6.3 Structura de date stivă

Structura de date stivă este introdusă într-o formă simplificată, apropiată de experiența fizică: utilizatorul așază obiecte într-o anumită ordine, iar validarea se bazează pe secvența obținută. Chiar dacă implementarea nu urmărește toate operațiile unei stive clasice, nivelul transmite ideea de ordine și de manipulare succesivă a elementelor.

3.6.4 Instrucțiunile condiționale

Instrucțiunile condiționale de tip if sunt prezentate printr-un mecanism de decizie bazat pe proprietăți vizibile, în cazul de față culoarea unor obiecte. Utilizatorul trebuie să

decidă corect între două variante, iar progresul depinde de evaluarea unei condiții. Această abordare transformă noțiunea abstractă de true / false într-o alegere concretă.

3.6.5 Structurile repetitive

Ultimul concept introdus este cel al structurilor repetitive. În joc, ideea este exprimată printr-un contor și prin necesitatea atingerii unei valori corecte pentru a activa rezultatul dorit. Astfel, utilizatorul poate înțelege mai ușor faptul că o buclă este legată de repetiție controlată și de condiția de oprire.

CAPITOLUL 4. SOLUȚIA PROPUȘĂ ȘI METODOLOGIA DE DEZVOLTARE

4.1 Cerințe funcționale

În etapa de proiectare au fost formulate câteva cerințe funcționale esențiale, care au ghidat atât structura aplicației, cât și felul în care au fost dezvoltate nivelurile.

- aplicația trebuie să permită deplasarea liberă a jucătorului într-un mediu 3D;
- fiecare nivel trebuie să fie încărcat și parcurs într-o ordine bine definită;
- rezolvarea corectă a nivelului trebuie să activeze accesul către următoarea zonă;
- aplicația trebuie să afișeze informații contextuale și feedback vizual;
- utilizatorul trebuie să poată înțelege scopul fiecărui nivel fără explicații excesiv de lungi;
- la finalizarea tuturor nivelurilor trebuie să existe un ecran final clar.

4.2 Cerințe nefuncționale

Pe lângă cerințele funcționale, proiectul a urmărit și câteva cerințe nefuncționale, importante pentru experiența generală a utilizatorului și pentru menținerea unei structuri coerente.

- controlul trebuie să fie simplu și stabil;
- mesajele informative trebuie să fie scurte și lizibile;
- feedback-ul trebuie să apară rapid după acțiunea utilizatorului;
- organizarea codului trebuie să permită extinderea aplicației;
- nivelurile trebuie să fie suficient de diferite între ele pentru a evita repetitivitatea.

4.3 Scenariul general de utilizare

Scenariul general presupune pornirea aplicației și plasarea utilizatorului într-o cameră inițială care introduce atmosfera jocului. De acolo, acesta începe să exploreze și să parcurgă pe rând nivelurile, fiecare fiind gândit ca un test aplicat pentru un anumit concept.

Utilizatorul poate consulta terminale informative atunci când are nevoie de context suplimentar, poate observa indiciile din mediul de joc și primește confirmări vizuale atunci când un nivel este completat. Acest flux a fost ales pentru a păstra un ritm natural și pentru a evita supraîncărcarea cu explicații textuale.

La final, după rezolvarea tuturor nivelurilor, jocul afișează un ecran de încheiere. Acest moment are rolul de a marca finalizarea parcursului și de a închide coerent experiența generală.

4.4 Fluxul aplicației

Fluxul aplicației a fost gândit astfel încât fiecare acțiune relevantă să aibă un sens clar în progresul general. Mai întâi este încărcată camera inițială, apoi managerul de nivel adaugă succesiv puzzle-urile asociate etapelor. În fiecare cameră există o singură sarcină principală, iar ieșirea rămâne blocată până la validarea acesteia.

Din punct de vedere al experienței de utilizare, acest lucru creează un echilibru util: utilizatorul are suficientă libertate de explorare cât să perceapă jocul ca pe un spațiu interactiv, dar nu atât de multă încât să piardă din vedere obiectivul educațional. Fiecare cameră își justifică existența prin conceptul pe care îl ilustrează.

4.5 Camera de spawn

Camera de spawn reprezintă punctul de pornire al experienței de joc. Această zonă a fost proiectată pentru a oferi utilizatorului un moment inițial de orientare, înainte ca acesta să intre în contact cu sarcinile educative propriu-zise. În această cameră sunt amplasate două panouri informative, poziționate pe pereții laterali, care introduc scopul aplicației și explică modul general de interacțiune.

Pe peretele din dreapta se află panoul de întâmpinare, care prezintă ideea principală a jocului. Acesta informează utilizatorul că mediul virtual a fost creat pentru testarea și consolidarea cunoștințelor de programare într-un mod interactiv. De asemenea, panoul menționează existența celor cinci camere de testare, fiecare construită în jurul unui concept fundamental din informatică, și recomandă analiza atentă a mediului pentru identificarea soluțiilor.

Pe peretele din stânga este amplasat panoul cu instrucțiuni de bază. Acesta prezintă comenzile principale ale jocului, precum deplasarea cu tastele W, A, S, D, rotirea camerei cu mouse-ul, săritura cu Space și interacțiunea cu obiectele prin click stânga. Totodată, panoul explică faptul că utilizatorul trebuie să apese butonul verde pentru deblocarea ușii și să mute obiectele necesare pentru rezolvarea puzzle-urilor.

Din punct de vedere al designului, camera de spawn nu are rolul de a încărca utilizatorul cu informații excesive, ci de a crea o tranziție clară între pornirea jocului și primul nivel. Poziționarea panourilor pe pereții laterali permite utilizatorului să consulte informațiile rapid, fără ca acestea să blocheze deplasarea către zona următoare.

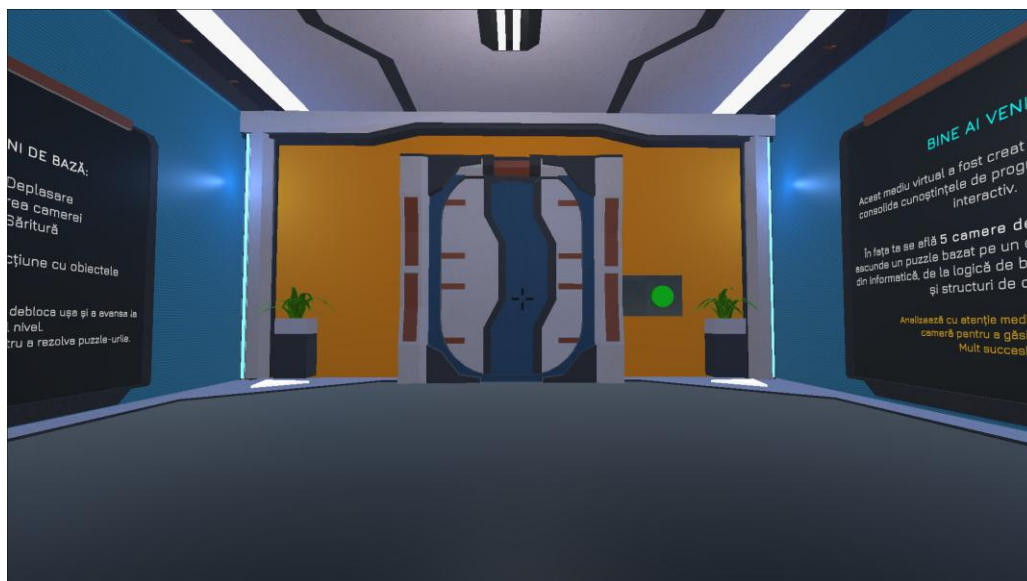


Figura 4.1. Camera de spawn

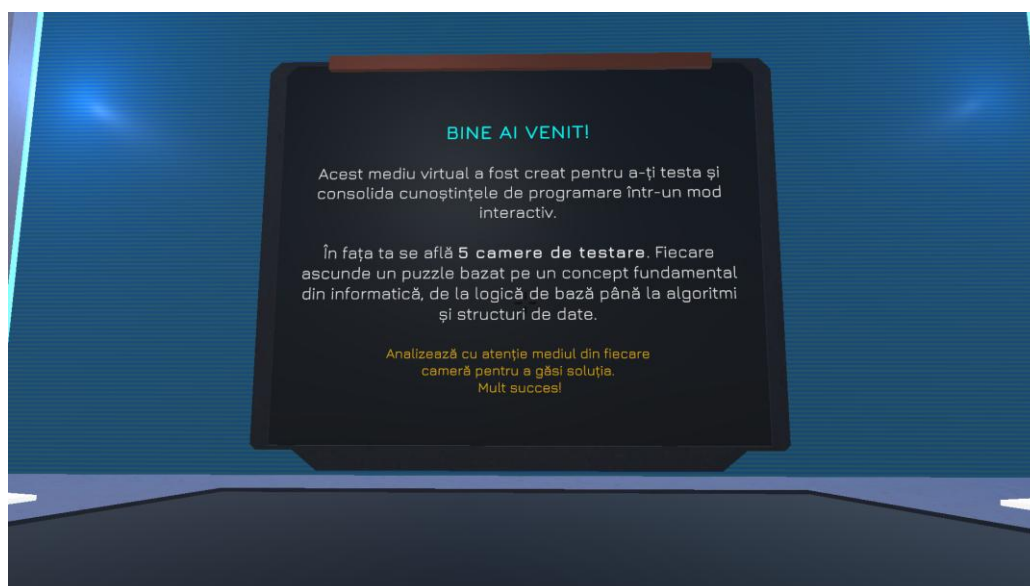


Figura 4.2. Panoul de întâmpinare din camera de spawn

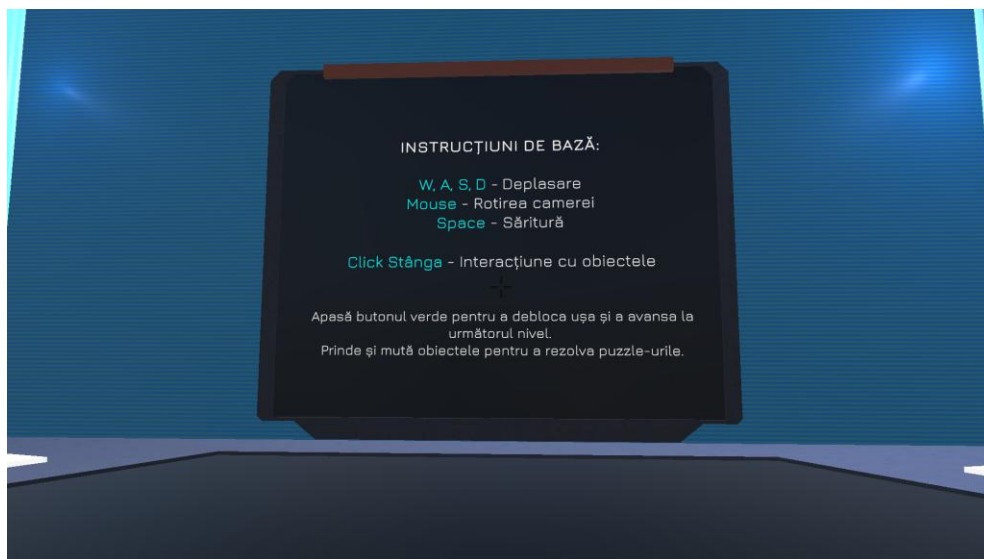


Figura 4.3. Panoul cu instrucțiuni de bază din camera de spawn

4.6 Proiectarea nivelului 1 – conversie binar-zecimal

Primul nivel a fost conceput pentru a introduce utilizatorul în conceptul de reprezentare a numerelor în sistem binar. Cerința nivelului constă în transformarea numărului zecimal 13 în forma sa binară, soluția corectă fiind 1101.

Nivelul folosește patru cuburi interactive, fiecare cub reprezentând o poziție binară. Fiecare cub poate avea una dintre cele două valori posibile, 0 sau 1, iar schimbarea stării sale influențează direct rezultatul obținut. Această reprezentare este potrivită pentru un prim contact cu un concept abstract, deoarece permite asocierea imediată dintre modificarea unei valori și efectul produs.

Din perspectiva proiectării, nivelul urmărește simplitatea și claritatea interacțiunii. Utilizatorul poate testa combinații diferite, poate observa rapid starea curentă a cuburilor și poate înțelege momentul în care a ajuns la configurația corectă. Validarea nivelului se realizează imediat ce valorile cuburilor corespund soluției stabilite.

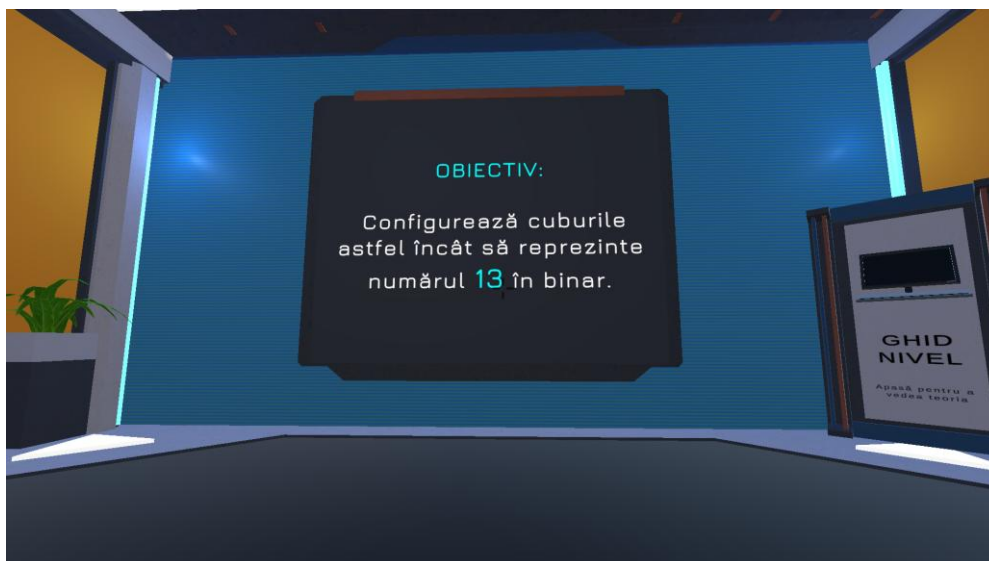


Figura 4.4. Cerința nivelului 1 privind reprezentarea numărului 13 în sistem binar

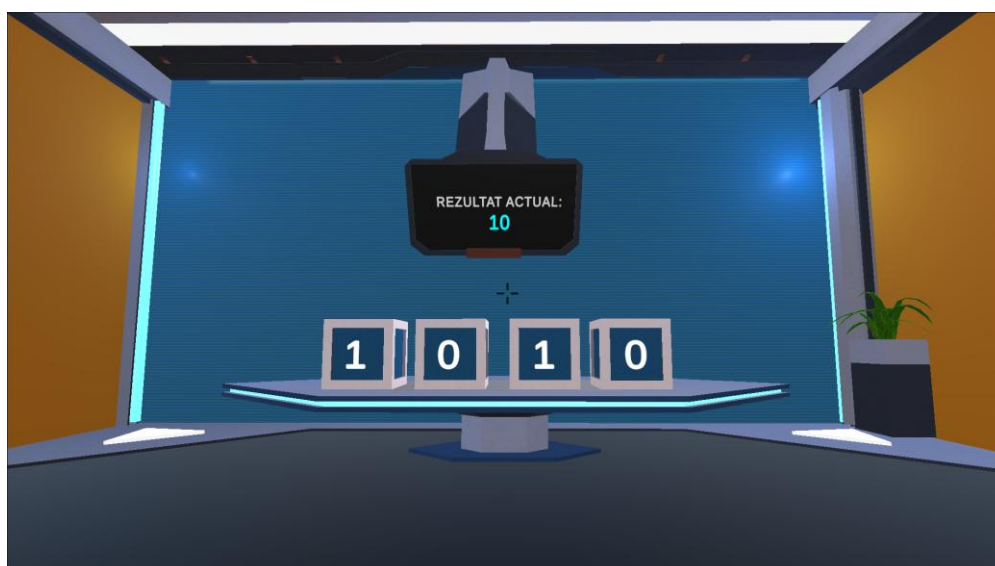


Figura 4.5. Cuburile interactive folosite pentru construirea valorii binare



Figura 4.6. Mesajul de validare afișat după rezolvarea corectă a nivelului 1

4.7 Proiectarea nivelului 2 – logică booleană

În cazul celui de-al doilea nivel, obiectivul a fost introducerea operatorilor logici printr-un puzzle bazat pe activarea unor baterii. Cerința nivelului este: „Activează reactorul. Setează corect bateriile A, B și C. Condiție de activare: $(A \text{ AND } B) \text{ OR } (\text{NOT } C)$.”

Utilizatorul interacționează cu trei baterii, notate cu A, B și C, fiecare putând avea valoarea 0 sau 1. Valoarea curentă este afișată deasupra fiecărei baterii, astfel încât combinația selectată să poată fi observată ușor. Expresia logică folosită în nivel este $(A \text{ AND } B) \text{ OR } (\text{NOT } C)$, unde AND este adevărat doar dacă ambele valori sunt 1, iar NOT inversează valoarea bateriei C.

În cameră se află un reactor și un buton de verificare. Dacă utilizatorul apasă butonul atunci când expresia logică nu este adevărată, reactorul este evidențiat cu roșu și este redat un sunet specific de eroare. Dacă expresia este corectă, reactorul începe să se rotească, luminile corespunzătoare se aprind, iar nivelul este validat. Astfel, utilizatorul înțelege relația dintre valorile binare ale bateriilor și rezultatul unei expresii logice.

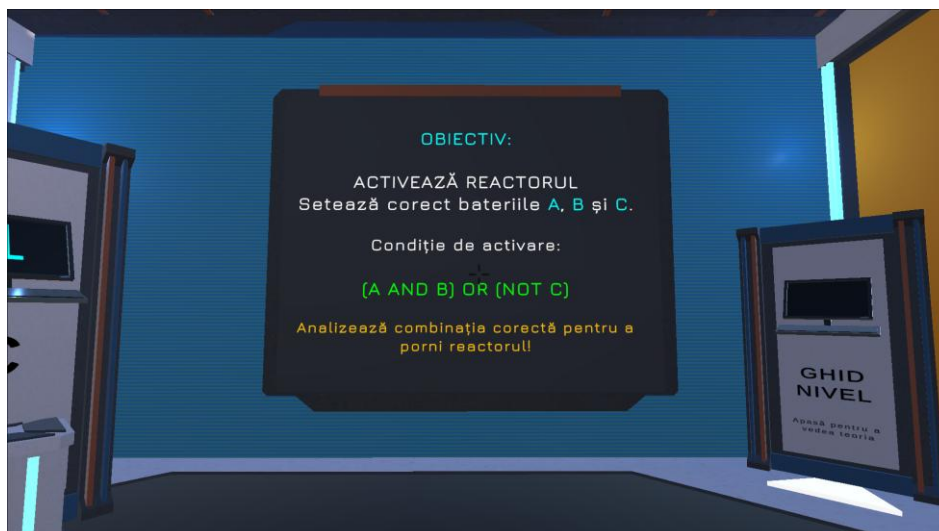


Figura 4.7. Cerința nivelului 2 privind expresia logică

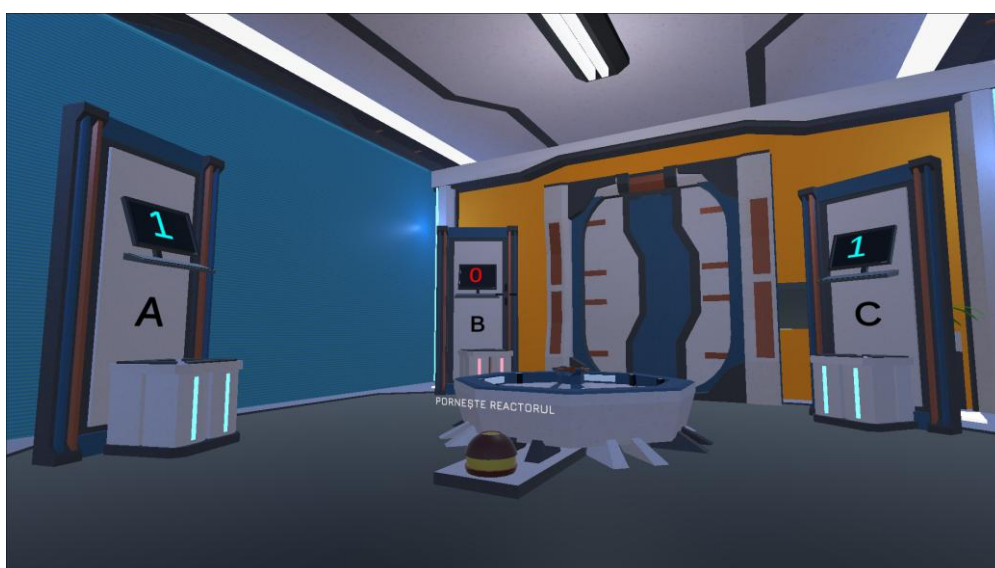


Figura 4.8. Bateriile interactive și valorile afișate deasupra acestora



Figura 4.9. Feedback-ul de eroare afișat la o combinație incorectă



Figura 4.10. Reactorul activat după rezolvarea corectă a nivelului 2

4.8 Proiectarea nivelului 3 – structură de tip stivă

În cazul celui de-al treilea nivel, obiectivul a fost introducerea structurii de tip stivă, cunoscută și sub denumirea de stack. Cerința nivelului este: „Ordinea containerelor este incorectă. Rearanjează containerele astfel încât stiva să respecte toate regulile.”

Regulile afișate în cameră sunt următoarele: containerul roșu nu trebuie plasat lângă cel albastru, stiva trebuie să înceapă cu un container verde, iar ultimul container trebuie să fie albastru. Utilizatorul trebuie să mute containerele colorate și să le așeze în zona de stivuire în ordinea corectă. În implementare, configurația validată este: Verde - Roșu - Verde - Albastru.

Nivelul folosește principiul LIFO (Last In, First Out), explicat printr-un panou informativ. Acest principiu înseamnă că ultimul obiect introdus în stivă este primul care poate

fi scos. Din acest motiv, utilizatorul poate muta doar containerul aflat în vârful stivei. Dacă încearcă să scoată un alt container, este afișat un mesaj de eroare și se redă un sunet specific.

Prin această mecanică, utilizatorul înțelege că ordinea elementelor este esențială într-o structură de tip stivă. Nivelul transformă conceptul teoretic într-o activitate vizuală și interactivă, în care regulile de plasare și restricția LIFO influențează direct rezolvarea puzzle-ului.

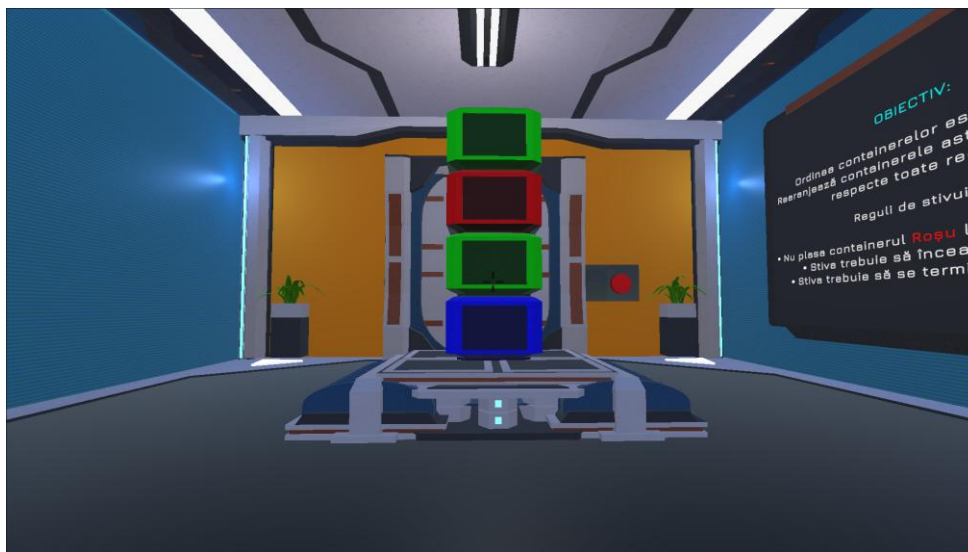


Figura 4.11. Intrarea în camera nivelului 3

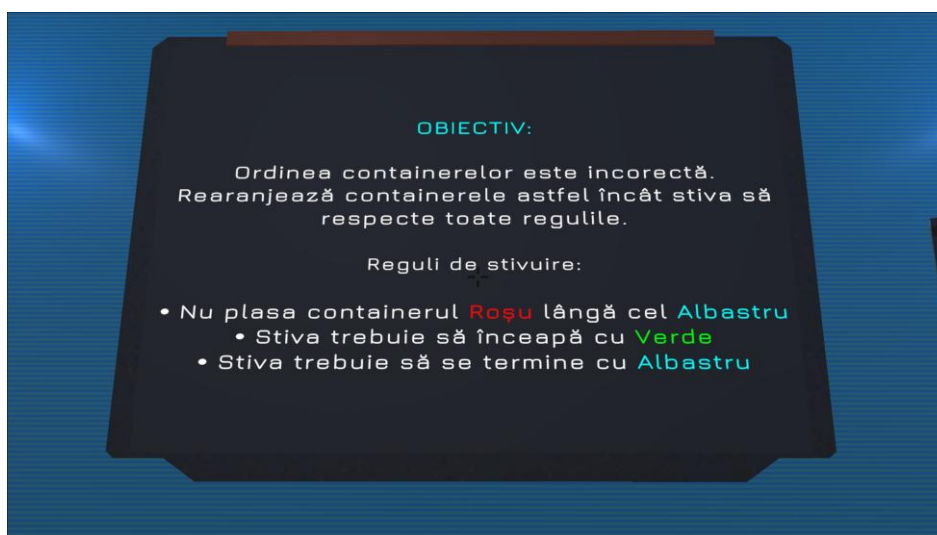


Figura 4.12. Cerința nivelului 3 privind rearanjarea containerelor



Figura 4.13. Configurația corectă a stivei

4.9 Proiectarea nivelului 4 – instrucțiuni condiționale

Al patrulea nivel a fost conceput pentru a ilustra modul în care o instrucțiune condițională poate decide rezultatul unei acțiuni. Utilizatorul urmărește sferele care ajung în zona de decizie și trebuie să stabilească dacă fiecare sferă respectă sau nu condiția afișată.

Cerința nivelului este: „Sortează sferele. Ai de sortat 4 sfere. Analizează algoritmul afișat deasupra benzii, decide dacă sfera respectă condiția, apasă TRUE dacă este corect și FALSE dacă nu este corect.” Algoritmul folosit este:

```
if (culoare == "Albastru" || culoare == "Roz") return true; else return false;
```

Astfel, utilizatorul trebuie să apese TRUE pentru sferele albastre sau roz și FALSE pentru celelalte culori. Progresul este afișat printr-un scor de tip 0/4, iar o greșeală resetează progresul, declanșează un feedback sonor specific și afișează un mesaj de avertizare pe ecran. Mesajul nu oferă direct soluția, ci îl îndeamnă pe utilizator să reanalizeze condiția IF, susținând astfel învățarea prin corectarea propriei alegeri. După patru răspunsuri corecte consecutive, nivelul este validat și accesul către următoarea cameră este deblocat.



Figura 4.14. Cerința nivelului 4 privind sortarea sferelor



Figura 4.15. Banda și algoritmul de sortare

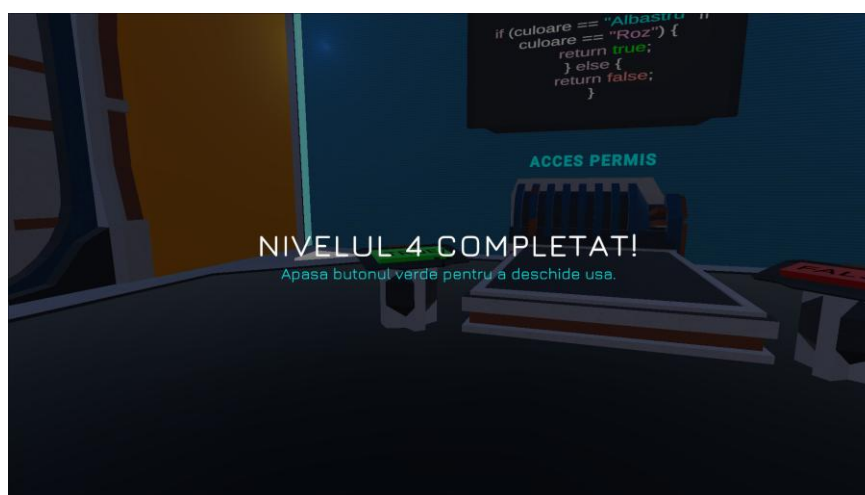


Figura 4.16. Mesajul de validare afișat după rezolvarea corectă a nivelului 4

4.10 Proiectarea nivelului 5 – structuri repetitive

Ultimul nivel a fost proiectat pentru a introduce ideea de structură repetitivă prin intermediul unei bucle de tip FOR. Utilizatorul trebuie să citească instrucțiunea afișată pe ecran și să introducă în container exact numărul de pachete cerut de buclă.

Cerința nivelului este: „Citește codul de pe ecranul containerului verde pentru a afla numărul de repetări. Aruncă acel număr exact de cuburi direct în container. Apasă EXECUTĂ când indexul i ajunge la valoarea corectă.” Codul afișat este:

```
for (int i = 0; i < 4; i++) { IncarcaPachet(i); }
```

Deoarece indexul pornește de la 0 și se oprește înainte de 4, acțiunea se repetă de patru ori. Prin urmare, soluția corectă este introducerea a exact 4 pachete în container. Dacă numărul este greșit, apare un mesaj de eroare, se redă un sunet specific și nivelul se resetează. Dacă numărul este corect, apăsarea butonului EXECUTĂ validează nivelul și marchează finalizarea parcursului.

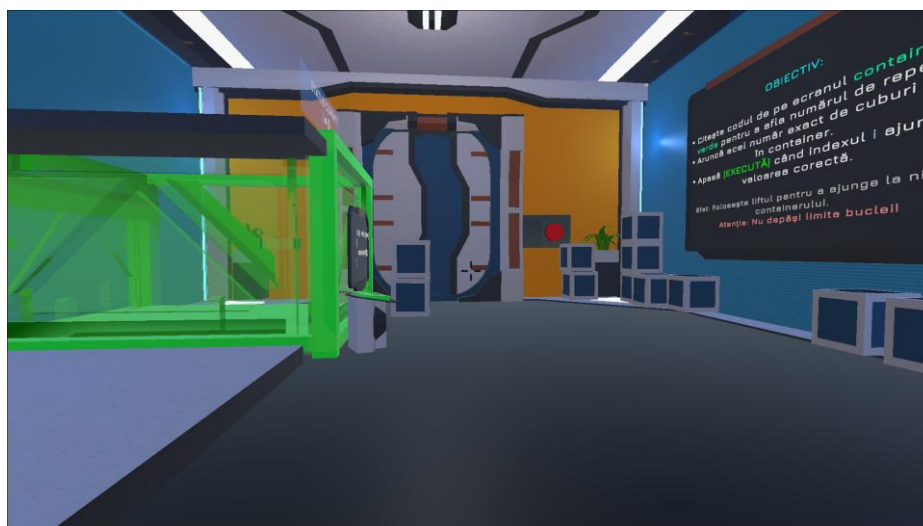


Figura 4.17. Intrarea în camera nivelului 5



Figura 4.18. Cerința nivelului 5 privind structura repetitivă



Figura 4.19. Containerul și algoritmul cu bucla FOR



Figura 4.20. Mesajul de validare finală

4.11 Elemente de interfață și feedback

Interfața aplicației a fost păstrată deliberat simplă, astfel încât utilizatorul să primească informații doar în momentele în care acestea sunt utile pentru înțelegerea nivelului sau pentru confirmarea rezultatului. În camera de start, o consolă suspendată amplasată deasupra zonei centrale afișează numele jocului, iar în camerele asociate

nivelurilor aceeași componentă este folosită pentru afișarea denumirii nivelului curent. În centrul ecranului este prezent și un reticul, utilizat ca reper vizual pentru interacțiunea cu obiectele și terminalele din joc.

Din punct de vedere vizual, textele interfeței au fost stilizate astfel încât să fie ușor de observat și de diferențiat. Pentru afișare a fost utilizat fontul Jura [9], iar în mesajele de feedback au fost folosite culori, caractere îngroșate și diferențe de dimensiune, pentru a evidenția informațiile importante. În acest mod, panourile informative, textele de stare și mesajele de finalizare completează acțiunea fără a încărca excesiv ecranul.

Feedback-ul vizual și audio joacă un rol important în orientarea utilizatorului. Acesta confirmă atât acțiunile corecte, cât și erorile, contribuind la menținerea unui ritm de învățare bazat pe încercare și corectare. Pentru componenta sonoră au fost utilizate efecte descărcate de pe site-ul Pixabay, din secțiunea Sound Effects [11], asociate unor acțiuni precum apăsarea butoanelor, plasarea obiectelor, semnalarea erorilor, deschiderea ușilor și finalizarea jocului.

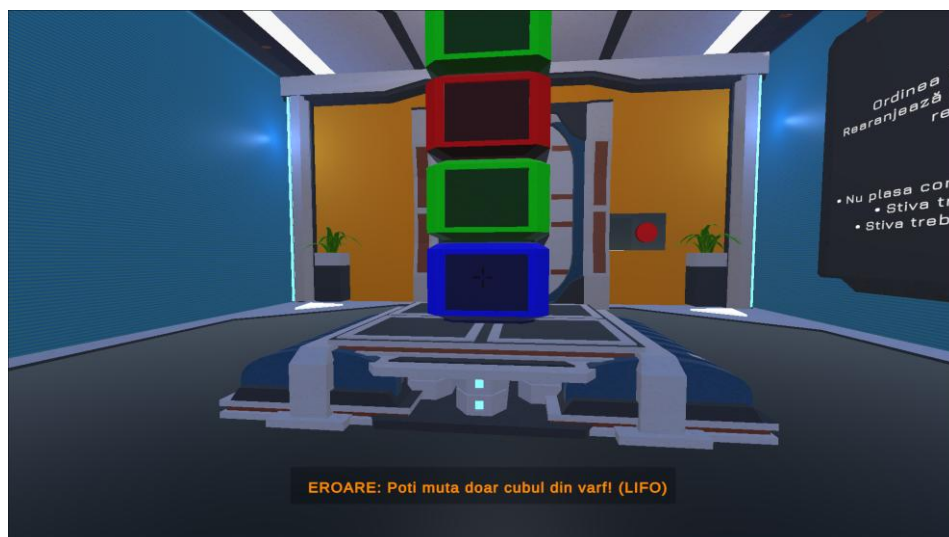


Figura 4.21. Feedback vizual la încălcarea regulii LIFO în nivelul 3



Figura 4.22. Feedback vizual la alegerea greșită în nivelul 4

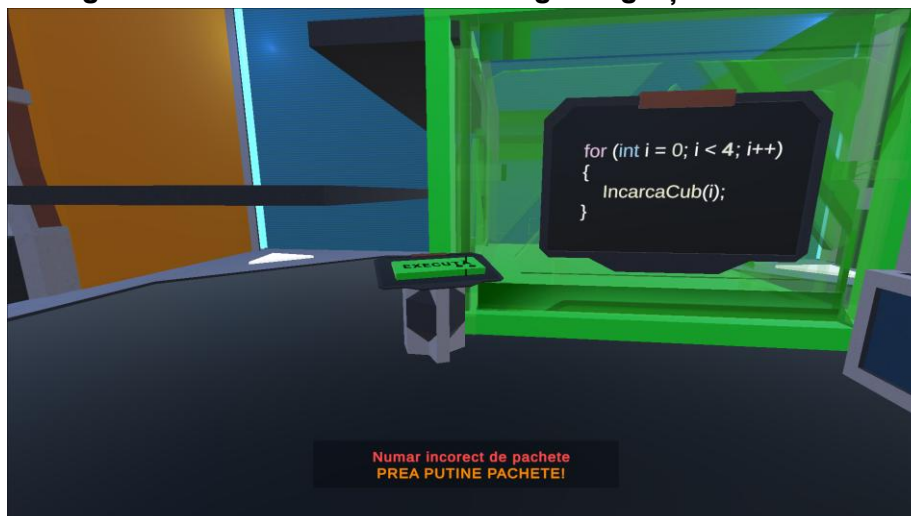


Figura 4.23. Feedback vizual la introducerea unui număr incorrect de pachete în nivelul 5

4.12 Metodologia de dezvoltare

Dezvoltarea aplicației a fost realizată incremental. Mai întâi a fost definită ideea generală și rolul educativ al fiecărui nivel, apoi au fost implementate mecanicile principale de deplasare, interacțiune și validare. Ulterior au fost adăugate feedback-ul vizual, mesajele de progres și ajustările necesare pentru coerența experienței de joc.

Această metodă a fost utilă deoarece a permis verificarea fiecărui nivel separat, înainte ca proiectul să fie considerat complet. În plus, a făcut mai ușor controlul relației dintre partea de design și partea tehnică: dacă un puzzle părea greu de înțeles, problema putea fi analizată atât la nivel de cod, cât și la nivel de prezentare vizuală.

Testarea a fost realizată prin parcurgerea repetată a scenariilor principale: deplasarea între camere, interacțiunea cu obiectele, verificarea răspunsurilor și tranziția către nivelul următor. Acest mod de lucru a permis identificarea situațiilor în care utilizatorul ar fi putut rămâne blocat sau ar fi primit feedback insuficient.

CAPITOLUL 5. IMPLEMENTAREA APLICAȚIEI CODESCAPE

5.1 Organizarea proiectului și structura generală

În ceea ce privește partea vizuală a mediului, proiectul folosește în proporție foarte mare asset pack-ul gratuit „Sci-Fi Styled Modular Pack” [10], din care provin majoritatea elementelor 3D utilizate în camere și în decorul general al aplicației. Alegerea acestuia a fost justificată de faptul că a oferit rapid un cadru vizual coerent pentru o aplicație cu tematică science-fiction și a permis orientarea timpului de dezvoltare spre logica aplicației, interacțiuni și designul nivelurilor.

Din punct de vedere al implementării, proiectul este organizat pe componente cu responsabilități relativ bine delimitate. Există clase dedicate controlului jucătorului, interfeței, managerului de nivel și logicii fiecărui puzzle. Această separare a ajutat la menținerea codului într-o formă mai ușor de urmărit și de extins.

Un avantaj important al acestei organizări este faptul că fiecare nivel poate fi explicat separat, fără a pierde însă legătura cu fluxul general al aplicației. Din perspectiva redactării lucrării, acest lucru este util deoarece permite asocierea clară dintre o funcționalitate vizibilă în joc și fragmentul de cod care o produce.

5.2 Implementarea controlului jucătorului

Mișcarea jucătorului este unul dintre elementele care susțin senzația de explorare într-un mediu 3D. În proiect, aceasta a fost implementată printr-un script dedicat care citește input-ul de deplasare, verifică dacă personajul se află la sol și aplică mișcarea în mod diferit în funcție de situație.

Soluția adoptată este suficient de simplă pentru a fi predictibilă și stabilă, dar și suficient de clară pentru a putea fi explicată direct. Jucătorul poate merge, sări și își poate controla direcția de deplasare, iar aceste mecanici creează baza pentru restul interacțiunilor din joc. În plus, viteza este limitată în script pentru a menține un comportament controlabil și constant în timpul deplasării.

```
1 reference
private void MyInput()
{
    horizontalInput = Input.GetAxisRaw("Horizontal");
    verticalInput = Input.GetAxisRaw("Vertical");

    if (Input.GetKeyDown(jumpKey) && grounded && readyToJump)
    {
        readyToJump = false;
        Jump();
        Invoke(nameof(ResetJump), jumpCooldown);
    }
}

1 reference
private void MovePlayer()
{
    moveDirection = orientation.forward * verticalInput + orientation.right * horizontalInput;

    if (grounded)
        rb.AddForce(moveDirection.normalized * movespeed * 10f, ForceMode.Force);
    else
        rb.AddForce(moveDirection.normalized * movespeed * 10f * airMultiplier, ForceMode.Force);
}
```

Figura 5.1. Fragment din scriptul PlayerMovement pentru citirea input-ului și deplasare

Metodele prezentate citesc input-ul utilizatorului pentru deplasarea pe axele orizontala si verticala, apoi calculeaza directia de miscare in functie de orientarea jucatorului. Forta de deplasare este aplicata asupra componentei Rigidbody, iar atunci cand personajul se afla in aer, aceasta este redusa prin variabila airMultiplier. Pe langa aceste metode, scriptul contine si metoda Jump(), care aplica impulsul de saritura, precum si metoda SpeedControl(), folosita pentru limitarea vitezei pe orizontala si mentinerea unui control stabil al personajului.

5.3 Gestionarea nivelurilor și a progresului

Coordonarea generală a aplicației este realizată de managerul de nivel. Această clasă are rolul de a încărca camera inițială, de a genera succesiv camerele nivelurilor, de a transmite butoanelor starea corectă și de a afișa mesajele de progres.

Importanța acestei componente este mare, deoarece ea leagă toate nivelurile într-un singur flux logic. Fără un astfel de manager, fiecare nivel ar rămâne un element izolat. În aplicație, însă, ele sunt tratate ca pași ai aceluiași parcurs educațional.

```
2 references
public void SpawnRoom(Transform spawnPoint, bool isSpawnRoom = false)
{
    if (!isSpawnRoom && nivelCurent >= levelPrefabs.Length)
    {
        Debug.Log("Toate nivelele au fost finalizate!");
        return;
    }

    GameObject roomCurenta = Instantiate(roomPrefab, spawnPoint.position, spawnPoint.rotation);
    RemoveBackWall(roomCurenta);

    LevelDoorButton levelButton = roomCurenta.GetComponentInChildren<LevelDoorButton>(true);
    if (levelButton != null) levelButton.SetInteractable(false);

    if (!isSpawnRoom)
    {
        SpawnLevelObjects(roomCurenta, levelButton);
        nivelCurent++;
    }

    UpdateScreenText(roomCurenta, isSpawnRoom);
}
```

Figura 5.2. Fragment din scriptul LevelManager pentru generarea camerelor

Fragmentul de cod evidențiază rolul metodei SpawnRoom() în gestionarea fluxului dintre niveluri. Aceasta verifică dacă mai există niveluri disponibile, generează camera curentă, dezactivează inițial butonul de trecere și, atunci când este cazul, încarcă obiectele specifice nivelului curent. Prin actualizarea textului afișat în cameră, metoda oferă utilizatorului feedback vizual despre progresul în aplicație.

```
1 reference
private void SpawnLevelObjects(GameObject roomCurenta, LevelDoorButton levelButton)
{
    GameObject nivelPrefab = levelPrefabs[nivelCurent];
    if (nivelPrefab == null) return;

    GameObject nivelObiecte = Instantiate(nivelPrefab, roomCurenta.transform);
    nivelObiecte.transform.localPosition = Vector3.zero;
    nivelObiecte.transform.localRotation = Quaternion.identity;

    if (levelButton == null) return;

    GameObject buttonObject = levelButton.gameObject;

    BitManager bitManager = nivelObiecte.GetComponentInChildren<BitManager>(true);
    if (bitManager != null) bitManager.SetupInteractButton(buttonObject);

    ReactorPuzzleManager reactorManager = nivelObiecte.GetComponentInChildren<ReactorPuzzleManager>(true);
    if (reactorManager != null) reactorManager.SetupInteractButton(buttonObject);

    StackPuzzleManager stackManager = nivelObiecte.GetComponentInChildren<StackPuzzleManager>(true);
    if (stackManager != null) stackManager.SetupInteractButton(buttonObject);

    IfPuzzleManager ifManager = nivelObiecte.GetComponentInChildren<IfPuzzleManager>(true);
    if (ifManager != null) ifManager.SetupInteractButton(buttonObject);

    LoopPuzzleManager loopManager = nivelObiecte.GetComponentInChildren<LoopPuzzleManager>(true);
    if (loopManager != null) loopManager.SetupInteractButton(buttonObject);
}
```

Figura 5.3. Fragment de cod Încărcarea obiectelor specifice nivelului curent

În această secvență este selectat prefab-ul asociat nivelului curent și este atașat camerei generate. De asemenea, butonul de interacțiune este transmis managerului de puzzle corespunzător, pentru a controla deblocarea trecerii către următorul nivel.

```
1 reference
private void UpdateScreenText(GameObject roomInstance, bool isSpawnRoom)
{
    TextMeshPro textComp = roomInstance.GetComponentInChildren<TextMeshPro>();
    if (textComp == null)
    {
        return;
    }

    if (isSpawnRoom)
    {
        textComp.text = $"<size=120>{gameName}</size>\n<size=60><color=#A0A0A0>Sistem de testare activat</color></size>";
        return;
    }

    textComp.text = $"LEVEL <color=#00FFFF>{nivelCurent}</color>\n<size=50><color=#A0A0A0>{GetLevelDescription()}</color></size>";
}
```

Figura 5.4. Actualizarea informațiilor afișate pe consola camerei

Această secvență stabilește textul afișat în cameră în funcție de contextul curent. Pentru camera inițială este afișat numele aplicației și un mesaj de pornire, iar pentru camerele de nivel sunt afișate numărul nivelului curent și descrierea acestuia.



Figura 5.5. Consola suspendată utilizată pentru afișarea denumirii nivelului

5.4 Terminale informative și panoul de informații

Pe lângă mecanicile efective ale nivelurilor, aplicația include și un sistem de terminale informative. Fiecare nivel conține un terminal propriu, căruia îi este asociat un text explicativ (infoText) referitor la conceptul de programare prezentat în puzzle-ul respectiv. Prin intermediul acestui mecanism, utilizatorul poate consulta rapid un scurt ghid teoretic fără a părăsi spațiul de joc, ceea ce susține componenta educațională a aplicației fără a încărca excesiv interfața.

Din punct de vedere tehnic, interacțiunea cu terminalele este realizată printr-un raycast lansat din camera jucătorului, din centrul ecranului, în momentul în care utilizatorul apasă click stânga. Dacă obiectul detectat este terminalul aflat în fața jucătorului și acesta se găsește în limita distanței de interacțiune, este afișat panoul informativ și este redat un efect sonor de deschidere. Conținutul ferestrei este gestionat de InfoPanelManager, care

actualizează textul, afișează panoul UI și face cursorul vizibil pentru citire. Închiderea panoului se poate realiza prin apăsarea tastei Escape, prin click stânga sau prin click dreapta, după care cursorul este ascuns din nou, iar controlul revine la modul normal de joc. Prin această soluție, partea teoretică este integrată direct în experiența first-person, fără a fi separată de interacțiunea propriu-zisă.

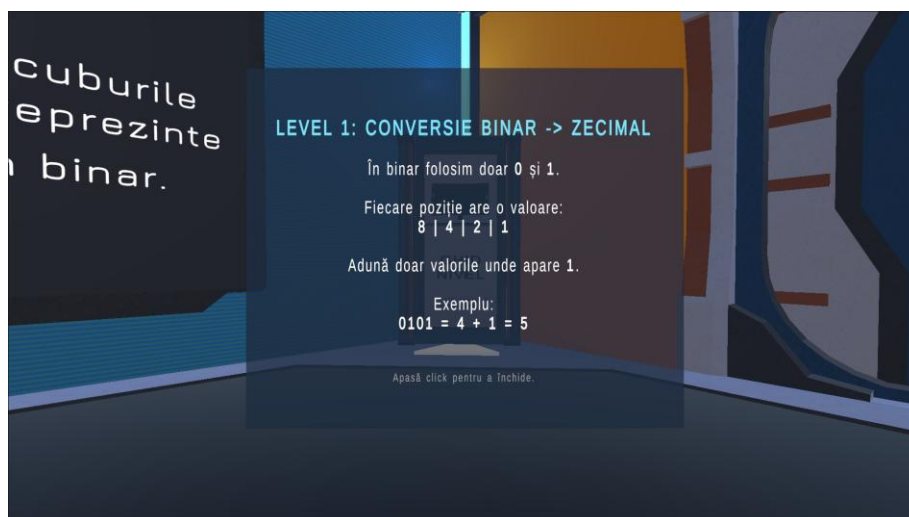


Figura 5.6. Panoul informativ utilizat pentru prezentarea ghidului teoretic de la nivelul 1

```
1 reference
private void TryOpenInfo()
{
    if (playerCamera == null || InfoPanelManager.instance == null)
    {
        return;
    }

    Ray ray = playerCamera.ScreenPointToRay(new Vector3(Screen.width / 2f, Screen.height / 2f, 0f));
    if (!Physics.Raycast(ray, out RaycastHit hit, interactDistance))
    {
        return;
    }

    if (hit.collider == null || hit.collider.gameObject != gameObject)
    {
        return;
    }

    if (openClip != null)
    {
        audioSource.PlayOneShot(openClip, openVolume);
    }

    InfoPanelManager.instance.ShowInfo(infoText);
}
```

Figura 5.7. Detectarea terminalului informativ prin raycast

Secvența verifică dacă utilizatorul privește către terminalul informativ și dacă acesta se află în raza de interacțiune. Dacă obiectul detectat este valid, este redat efectul sonor de deschidere, iar textul explicativ este transmis către panoul informativ.

```
1 reference
public void ShowInfo(string infoText)
{
    if (!CanOpenInfo)
    {
        return;
    }

    if (panelRoot == null)
    {
        Debug.LogWarning("InfoPanelManager: panelRoot nu este setat.");
        return;
    }

    if (contentText != null)
    {
        contentText.text = infoText;
    }

    panelRoot.SetActive(true);
    isOpen = true;
    Cursor.lockState = CursorLockMode.None;
    Cursor.visible = true;
}
```

Figura 5.8. Afișarea panoului informativ

Secvența actualizează conținutul panoului cu textul explicativ primit, apoi activează interfața informativă. Cursorul este deblocat și devine vizibil pentru citire, iar la închiderea panoului acesta este ascuns din nou, cursorul este blocat, iar controlul revine la modul normal de joc.

5.5 Implementarea nivelului 1 – reprezentarea binară

Nivelul 1 validează configurarea corectă a unor cuburi binare. Fiecare cub are o stare, iar managerul nivelului calculează valoarea curentă și verifică dacă toate pozițiile corespund soluției. Implementarea este simplă, dar relevantă, deoarece transformă o combinație vizuală într-un rezultat numeric.

Din punct de vedere didactic, acest nivel funcționează bine tocmai pentru că este ușor de urmărit. Utilizatorul poate observa schimbarea valorii și poate înțelege faptul că ordinea și starea fiecărui bit influențează rezultatul final.

```
2 references
public int GetValue()
{
    if (bitCubes == null || bitCubes.Length == 0)
    {
        return 0;
    }

    int value = 0;

    for (int i = 0; i < bitCubes.Length; i++)
    {
        int bit = 0;
        if (bitCubes[i] != null)
        {
            bit = Mathf.Clamp(bitCubes[i].bitValue, 0, 1);

            value = (value << 1) | bit;
        }
    }

    return value;
}
```

Figura 5.9. Calcularea valorii binare pe baza cuburilor active

Secvența parcurge cuburile binare și construiește valoarea numerică finală prin deplasarea rezultatului la stânga și adăugarea bitului curent. Astfel, starea vizuală a cuburilor este transformată într-o valoare numerică ușor de verificat.ⁱ

```
2 references
public void CheckAllBits()
{
    if (nivelCompletat)
    {
        return;
    }

    foreach (BitCube cube in bitCubes)
    {
        if (cube == null)
        {
            continue;
        }

        if (cube.bitValue != cube.correctBit)
        {
            return;
        }
    }

    nivelCompletat = true;
    SetButtonState(true);
    LevelManager.instance.ShowLevelCompleteMessage();
}
```

Figura 5.10. Validarea configurației binare corecte

Secvența compară valoarea fiecărui cub cu bitul corect stabilit pentru soluție. Dacă toate pozițiile sunt configurate corect, nivelul este marcat ca finalizat, butonul de trecere devine activ și este afișat mesajul de confirmare.

5.6 Implementarea nivelului 2 – logică booleană

Al doilea nivel introduce o verificare logică între mai multe elemente active sau inactive. Din punct de vedere al implementării, soluția este elegantă deoarece expresia este relativ scurtă, dar reflectă clar regula din spatele puzzle-ului.

Un alt aspect important este feedback-ul vizual. Dacă soluția este corectă, reactorul se activează și se produce o schimbare perceptibilă în scenă. Dacă utilizatorul greșește, primește un răspuns negativ temporar, suficient de clar pentru a înțelege că trebuie să încerce din nou.

```
public void CheckSolution()
{
    if (isSolved) return;

    bool a = IsBatteryOn(batteryA);
    bool b = IsBatteryOn(batteryB);
    bool c = IsBatteryOn(batteryC);

    if ((a && b) || !c)
    {
        SolvePuzzle();
        return;
    }

    PlayClip(errorClip, errorVolume);
    StartCoroutine(FlashErrorEffect());
}

1 reference
private void SolvePuzzle()
{
    isSolved = true;
    PlayClip(successClip, successVolume);

    if (projectorRenderer != null)
    {
        Material[] mats = projectorRenderer.materials;
        SetEmission(mats, WhiteMaterialIndex, Color.white, 4f);
        SetEmission(mats, CyanMaterialIndex, Color.cyan, 5f);
        projectorRenderer.materials = mats;
    }

    if (interactButton != null)
    {
        interactButton.SetInteractable(true);
    }

    LevelManager.instance.ShowLevelCompleteMessage();
}
```

Figura 5.11. Validarea configurației binare corecte

În cadrul verificării, stările celor trei baterii sunt citite ca valori booleene și sunt evaluate printr-o expresie logică ce stabilește succesul sau eșecul puzzle-ului. Această legătură este relevantă din punct de vedere didactic, deoarece utilizatorul poate observa direct cum o condiție logică produce un rezultat concret în scenă.

În cazul unei alegeri greșite, este redat un sunet de eroare, iar reactorul afișează temporar un feedback negativ prin colorarea luminilor în roșu. Dacă expresia este evaluată corect, reactorul își modifică emisia luminoasă, butonul de progres este deblocat, iar nivelul este marcat ca finalizat. Astfel, implementarea nu produce doar un rezultat intern, ci generează o consecință clară și ușor de interpretat în joc.

5.7 Implementarea nivelului 3 – structură de tip stivă

Nivelul 3 urmărește ordinea în care sunt așezate cuburile și validează secvența obținută după completarea stivei. Implementarea se bazează pe două liste: una pentru obiectele efective, alta pentru culorile lor, astfel încât verificarea finală să poată fi făcută simplu.

Un aspect important al interacțiunii este faptul că utilizatorul poate muta doar cubul aflat în vârful stivei, în acord cu principiul LIFO. Dacă încearcă să selecteze un alt cub, acțiunea este blocată, iar jocul oferă un mesaj de avertizare corespunzător.

Pe lângă logica de validare, nivelul folosește și elemente de fizică pentru obiectele manipulate. Odată adăugate în stivă, acestea sunt repositionate, devin kinematice și nu mai pot fi tratate ca obiecte libere, ceea ce întărește senzația că au fost integrate într-o structură ordonată.

Pentru a menține comportamentul stabil al nivelului, cuburile introduse în stivă sunt poziționate controlat în zona de plasare și nu rămân dependente exclusiv de simularea fizică. În acest mod, ordinea obținută poate fi verificată corect, iar interacțiunea rămâne predictibilă pentru utilizator.

```
if (hit.transform.CompareTag(Level3Tag))
{
    StackItem cubeInfo = hit.transform.GetComponent<StackItem>();
    if (cubeInfo != null && cubeInfo.isStacked)
    {
        StackPuzzleManager stackBase = FindObjectOfType<StackPuzzleManager>();
        if (stackBase != null)
        {
            if (!stackBase.IsTopCube(hit.transform.gameObject))
            {
                stackBase.PlayErrorSound();
                if (warningText != null)
                    ShowWarningMessage("EROARE: Poti muta doar cubul din varf! (LIFO)");

                return;
            }

            stackBase.RemoveCubeFromStack(hit.transform.gameObject);
        }
    }
}
```

Figura 5.12. Restricționarea interacțiunii la cubul din vârful stivei

Secvența verifică dacă obiectul selectat aparține stivei și permite mutarea doar dacă acesta se află în vârful structurii. În cazul selectării unui alt cub, acțiunea este blocată, este redat un sunet de eroare și este afișat un mesaj de avertizare, respectând principiul LIFO.

```
2 references
private void AddCubeToStack(GameObject cubeObj, StackItem stackItem, Rigidbody rb)
{
    if (currentStack.Count >= maxStackSize)
    {
        return;
    }

    rb.isKinematic = true;
    stackItem.isStacked = true;

    Vector3 snapPos = transform.position + Vector3.up * currentStack.Count * cubeHeight;
    cubeObj.transform.position = snapPos;
    cubeObj.transform.rotation = Quaternion.identity;

    currentStack.Add(cubeObj);
    currentColorStack.Add(stackItem.cubeColor);
    PlayClip(placeClip, placeVolume);
    CheckWinCondition();
}
```

Figura 5.13. Adăugarea cuburilor în stivă

Secvența fixează cubul plasat, îl poziționează în funcție de numărul elementelor existente și îl adaugă în listele interne ale stivei. După plasare, cubul devine kinematic, iar ordinea curentă este verificată pentru a determina dacă nivelul a fost rezolvat.

```
1 reference
private void CheckWinCondition()
{
    if (currentColorStack.Count != targetSequence.Count)
    {
        return;
    }

    for (int i = 0; i < targetSequence.Count; i++)
    {
        if (currentColorStack[i].ToLower() != targetSequence[i].ToLower())
        {
            return;
        }
    }

    if (levelButton != null)
    {
        levelButton.SetInteractable(true);
    }

    LevelManager.instance.ShowLevelCompleteMessage();
}
```

Figura 5.14. Verificarea ordinii elementelor din stivă

Secvența compară ordinea culorilor din stiva curentă cu secvența țintă definită pentru nivel. Dacă toate elementele coincid, butonul de progres este activat, iar nivelul este marcat ca finalizat prin afișarea mesajului de confirmare.

5.8 Implementarea nivelului 4 – decizii condiționale

Nivelul 4 este organizat în jurul unei serii de decizii true / false. Utilizatorul trebuie să interpreteze corect culoarea sferei curente și să aleagă răspunsul potrivit. Sferele sunt generate succesiv, în mod aleator, dintr-un set restrâns de culori, iar fiecare nouă apariție obligă utilizatorul să aplice din nou aceeași regulă de decizie.

În plus, nivelul introduce ideea de progres bazat pe consecvență. Pentru finalizare nu sunt suficiente patru răspunsuri corecte în total, ci este necesară obținerea a patru răspunsuri corecte consecutive. Dacă utilizatorul greșește, scorul este resetat, se redă un sunet de eroare și este afișat un mesaj de avertizare pe ecran. Mesajul îl îndeamnă pe utilizator să reanalizeze condiția IF, fără a-i oferi direct soluția. Această regulă face experiența mai dinamică și mai apropiată de o condiție evaluată repetat.

```
1 reference
public void OnPlayerDecision(bool pressedTrue)
{
    if (levelCompleted || currentSphere == null)
    {
        return;
    }

    bool raspunsCorect = currentColor == Color.blue || currentColor == colorPink;

    if (pressedTrue == raspunsCorect)
    {
        score++;
    }
    else
    {
        score = 0;
        PlayClip(wrongClip, wrongVolume);
        ShowWrongMessage();
    }

    if (statusText != null)
    {
        statusText.text = "SCOR: " + score + "/4";
    }

    if (score >= 4)
    {
        FinishLevel();
        return;
    }

    Destroy(currentSphere);
    SpawnNewSphere();
}
```

Figura 5.15. Validarea deciziilor condiționale

Secvența evaluează culoarea sferei curente și compară rezultatul cu răspunsul ales de utilizator. Pentru fiecare decizie corectă scorul crește, iar în cazul unei greșeli scorul este resetat și este afișat feedback-ul de avertizare.

```
1 reference
private void FinishLevel()
{
    levelCompleted = true;
    isMoving = false;

    if (statusText != null)
    {
        statusText.text = "ACCES PERMIS";
    }

    Destroy(currentSphere);
    currentSphere = null;

    if (levelButton != null)
    {
        levelButton.SetInteractable(true);
    }

    LevelManager.instance.ShowLevelCompleteMessage();
}
```

Figura 5.16. Finalizarea nivelului după patru răspunsuri corecte consecutive

Secvența marchează nivelul ca finalizat după atingerea scorului necesar, oprește generarea sferei și actualizează mesajul de stare. În același timp, butonul de progres devine interactiv, permițând trecerea către următorul nivel.

5.9 Implementarea nivelului 5 – structuri repetitive și contorizare

Ultimul nivel verifică dacă utilizatorul a introdus exact numărul corect de pachete pentru a atinge limita buclei. Din punct de vedere al implementării, acesta este unul dintre cele mai interesante niveluri, deoarece include atât condiția de succes, cât și un mecanism explicit de eroare și resetare.

Valoarea contorului este actualizată în funcție de prezența pachetelor în zona containerului, iar utilizatorul poate observa permanent valoarea curentă a lui *i*. În practica nivelului, pachetele sunt introduse succesiv în container până la atingerea condiției cerute, iar în cazul unui număr greșit jocul afișează mesajul corespunzător și pornește secvența de resetare. În acest mod, legătura dintre acțiune, contor și rezultat rămâne clară pentru utilizator.

La acest nivel au fost introduse și măsuri suplimentare pentru evitarea unor situații nedorite în timpul interacțiunii. Pachetele de date sunt readuse în pozițiile inițiale în momentul resetării, astfel încât utilizatorul să poată relua nivelul în condiții identice. În plus, zona containerului este controlată prin coliziuni auxiliare, pentru a împiedica introducerea obiectelor prin laterale sau pătrunderea jucătorului în interiorul acestuia. Tot din motive de siguranță, liftul verifică existența jucătorului sub platformă înainte de coborâre, evitând astfel o situație în care acesta ar putea fi blocat sau prins între obiecte.

```
1 reference
public void ApasaExecute()
{
    if (i == limitaBucla)
    {
        lift?.OpresteLiftul();
        levelButton?.SetInteractable(true);
        LevelManager.instance.ShowLevelCompleteMessage();
        return;
    }

    if (errorClip != null)
    {
        audioSource.PlayOneShot(errorClip, errorVolume);
    }

    string mesajEroare = i < limitaBucla ? "PREA PUTINE PACHETE!" : "PREA MULTE PACHETE!";
    StartCoroutine(EroareSiReset(mesajEroare));
}
```

Figura 5.17. Verificarea contorului nivelului repetitiv

Secvența compară valoarea curentă a contorului cu limita stabilită pentru buclă. Dacă valorile coincid, liftul este oprit, butonul de progres devine interactiv și nivelul este marcat ca finalizat; în caz contrar, este afișat un mesaj de eroare corespunzător.

```
1 reference
private IEnumerator EroareSiReset(string mesaj)
{
    if (feedbackPanel != null)
    {
        feedbackPanel.SetActive(true);
    }

    if (feedbackText != null)
    {
        feedbackText.text = $"<color=#FF4C4C>Numar incorect de pachete</color>\n{mesaj}";
    }

    yield return new WaitForSeconds(2.5f);

    if (feedbackText != null)
    {
        feedbackText.text = "<color=#FFEB04>Se reseteaza nivelul...</color>";
    }

    yield return new WaitForSeconds(1.5f);

    i = 0;
    ActualizeazaEcran();

    if (feedbackText != null)
    {
        feedbackText.text = "";
    }

    if (feedbackPanel != null)
    {
        feedbackPanel.SetActive(false);
    }
}
```

Figura 5.18. Afișarea feedback-ului și resetarea contorului

Secvența afișează mesajul de eroare atunci când numărul de pachete introdus nu corespunde limitei cerute. După un scurt interval, utilizatorul este anunțat că nivelul se resetează, contorul este readus la zero, iar panoul de feedback este ascuns pentru reluarea încercării.

```
for (int index = 0; index < pacheteInScena.Count; index++)  
{  
    GameObject pachet = pacheteInScena[index];  
    Rigidbody rb = pachet.GetComponent<Rigidbody>();  
  
    if (rb != null && !rb.isKinematic)  
    {  
        rb.velocity = Vector3.zero;  
        rb.angularVelocity = Vector3.zero;  
    }  
  
    pachet.transform.position = pozitiiInitiale[index];  
    pachet.transform.rotation = rotatiiInitiale[index];  
}
```

Figura 5.19. Repoziționarea pachetelor după resetare

Secvența parcurge toate pachetele existente în scenă, oprește mișcarea acestora și le readuce la pozițiile și rotațiile inițiale. Astfel, nivelul poate fi reluat în aceleași condiții după o încercare greșită.

5.10 Finalizarea jocului și ecranul final

După rezolvarea ultimului nivel, aplicația afișează ecranul final și schimbă starea cursorului, astfel încât utilizatorul să poată interacționa cu opțiunile disponibile. În această etapă, utilizatorul poate alege fie repornirea jocului prin opțiunea Restart, fie închiderea aplicației prin opțiunea Quit. Închiderea progresului este importantă deoarece marchează clar trecerea de la experiența de joc la o stare finală a aplicației.

Existența unor opțiuni precum Restart și Quit contribuie la impresia de produs complet, nu doar de demonstrație tehnică. Chiar dacă aceste componente sunt mai simple decât logica nivelurilor, ele fac parte din experiența generală a utilizatorului și merită menționate în documentație.



Figura 5.20. Ecran final

```
1 reference
public bool IsGameComplete()
{
    return nivelCurent >= levelPrefabs.Length;
}

1 reference
public void ShowFinalGameScreen()
{
    if (finalPanel == null)
    {
        Debug.LogError("Final Panel nu este asignat in Inspector!");
        return;
    }

    finalPanel.SetActive(true);
    PlayFinalCompleteSound();

    if (levelCompletePanel != null)
    {
        levelCompletePanel.SetActive(false);
    }

    Cursor.lockState = CursorLockMode.None;
    Cursor.visible = true;
}

0 references
public void RestartGame()
{
    SceneManager.LoadScene(SceneManager.GetActiveScene().name);
}

0 references
public void QuitGame()
{
    Application.Quit();
    Debug.Log("Jocul s-a inchis.");
}
```

Figura 5.21. Afișarea ecranului final și gestionarea opțiunilor de ieșire

Secvența verifică dacă toate nivelurile au fost parcurse, apoi activează panoul final și deblochează cursorul pentru interacțiunea cu meniul. Utilizatorul poate reporni scena curentă prin opțiunea Restart sau poate închide aplicația prin opțiunea Quit.

CAPITOLUL 6. TESTAREA APLICAȚIEI

6.1 Strategia de testare

Testarea aplicației a urmărit două direcții principale: verificarea funcționării tehnice și validarea experienței de utilizare. Din punct de vedere tehnic, a fost important ca fiecare nivel să răspundă corect la acțiunile utilizatorului, iar din punct de vedere al experienței a fost esențial ca obiectivul fiecărui nivel să fie suficient de clar.

Deoarece proiectul este un joc educațional, testarea nu poate fi redusă doar la identificarea de erori. Ea trebuie să urmărească și dacă feedback-ul este suficient, dacă utilizatorul înțelege ce are de făcut și dacă trecerea de la un nivel la altul se produce natural.

6.2 Teste funcționale

Tabelul 6.1. Scenarii de testare funcțională

ID	Scenariu	Pași principali	Rezultat așteptat	Rezultat obținut
T1	Pornirea jocului	Lansarea scenei principale	Camera de start este încărcată corect	Conform așteptărilor
T2	Mișcarea jucătorului	Utilizarea tastelor W, A, S, D	Jucătorul se deplasează fără blocaje	Conform așteptărilor
T3	Săritura	Apăsarea tastei Space	Jucătorul execută o săritură unică și controlată	Conform așteptărilor
T4	Terminal informativ	Apropiere și click pe terminal	Panoul cu informații se deschide	Conform așteptărilor
T5	Închidere terminal	Apăsare Escape sau click	Panoul se închide, iar controlul revine jucătorului	Conform așteptărilor
T6	Nivelul 1	Setarea corectă a cuburilor binare	Se activează butonul de ieșire	Conform așteptărilor
T7	Nivelul 2	Configurarea corectă a bateriilor	Reactorul se activează și se deblochează progresul	Conform așteptărilor
T8	Nivelul 3 – protecție obiecte ținute în mână	Încercarea de a arunca un cub prin perete sau prin podea	Cubul nu este lăsat să traverseze obstacolele și rămâne într-o poziție controlată	Conform așteptărilor
T9	Nivelul 3	Așezarea cuburilor în ordinea cerută	Stiva este validată și nivelul este completat	Conform așteptărilor
T10	Nivelul 3 - regulă LIFO	Încercarea de a selecta un cub care nu se află în vârful stivei	Acțiunea este blocată, se redă sunetul de eroare și se afișează mesajul de avertizare	Conform așteptărilor

T11	Nivelul 4	Selectarea corectă a patru răspunsuri consecutive	Scorul ajunge la 4/4 și nivelul se finalizează	Conform așteptărilor
T12	Nivelul 4 - răspuns greșit	Alegerea unei variante incorecte pentru sfera curentă	Scorul se resetează, se redă sunetul de eroare și se afișează mesajul de avertizare	Conform așteptărilor
T13	Nivelul 5	Introducerea numărului corect de pachete	Liftul se oprește și nivelul este marcat ca finalizat	Conform așteptărilor
T14	Nivelul 5 – eroare	Executarea cu număr greșit de pachete	Se afișează mesaj de eroare și se resetează nivelul	Conform așteptărilor
T15	Nivelul 5 – protecție lift	Poziționarea jucătorului sub lift în timpul coborârii	Liftul nu coboară peste jucător	Conform așteptărilor
T16	Nivelul 5 – protecție container	Încercarea de a introduce pachetele prin lateral sau de a intra în container	Interacțiunea este limitată de coliziuni, iar comportamentul rămâne controlat	Conform așteptărilor
T17	Finalizarea jocului	Parcurgerea tuturor nivelurilor	Se afișează ecranul final	Conform așteptărilor

6.3 Probleme întâmpinate și remedieri

În timpul dezvoltării au apărut mai multe probleme legate de controlul fizic al obiectelor și de comportamentul jucătorului în anumite situații-limită. Nu a fost suficientă doar implementarea mecanicilor de bază, ci a fost necesară și tratarea unor cazuri în care obiectele puteau ajunge în poziții incorecte sau utilizatorul putea ocoli mecanica propusă.

Un exemplu relevant a apărut la nivelurile 3 și 5, unde au fost necesare măsuri suplimentare pentru manipularea obiectelor. În cazul nivelului 3, a trebuit controlată nu doar ordinea din stivă, ci și modul în care cuburile sunt selectate și eliberate, astfel încât utilizatorul să poată muta doar elementul aflat în vârf, iar obiectele ținute în mână să nu poată fi aruncate prin podea, prin pereți sau în alte zone nepermise. În cazul nivelului 5 au fost adăugate și limite fizice pentru container, astfel încât utilizatorul să nu poată introduce obiectele prin lateral și nici să nu poată sări în interiorul acestuia.

O altă problemă a fost legată de liftul din ultimul nivel. Mișcarea acestuia a trebuit ajustată pentru a evita situațiile în care jucătorul ar putea fi prins sub platformă, împins într-o poziție nefavorabilă sau scos din zona normală de joc. Din acest motiv, a fost introdus un mecanism suplimentar de verificare, astfel încât liftul să nu coboare peste jucător. În plus, la introducerea unui număr greșit de pachete, nivelul resetează obiectele în pozițiile inițiale, pentru ca utilizatorul să poată relua încercarea în condiții identice.

Probleme similare au apărut și în cazul terminalelor informative, unde a fost necesară gestionarea atentă a cursorului și a modului de închidere a panoului, pentru a evita comportamente neclare și pentru a asigura revenirea corectă la modul normal de joc.

Aceste ajustări au arătat că, într-o aplicație de acest tip, funcționalitatea și experiența utilizatorului sunt strâns legate. O implementare corectă din punct de vedere tehnic poate fi totuși insuficientă dacă utilizatorul nu înțelege clar ce s-a întâmplat sau dacă apar situații care afectează controlul normal al jocului.

6.4 Observații privind experiența de utilizare

Din punct de vedere calitativ, aplicația are avantajul unui parcurs clar și al unor niveluri diferențiate. Faptul că fiecare cameră corespunde unei singure idei ajută la evitarea confuziei. În plus, feedback-ul imediat contribuie la menținerea unui ritm bun de învățare.

Există însă și limite naturale. Unele concepte sunt simplificate pentru a putea fi transpuse în joc, iar această simplificare este utilă didactic, dar nu poate acoperi întreaga complexitate teoretică. Totuși, pentru obiectivul asumat al proiectului, această alegere este justificată.

6.5 Limite actuale ale aplicației

În forma actuală, aplicația se concentrează pe un set restrâns de concepte introductive și nu include un sistem de salvare a progresului, un nivel de dificultate adaptiv sau o evaluare mai detaliată a performanței utilizatorului. De asemenea, explicațiile oferite prin terminale sunt concise și pot fi extinse ulterior.

Aceste limite nu reduc utilitatea aplicației ca prototip educațional, dar indică în mod clar direcțiile în care proiectul poate fi dezvoltat. Din perspectiva lucrării, ele sunt importante pentru a delimita corect ceea ce a fost realizat și ceea ce rămâne deschis pentru etape viitoare.

CAPITOLUL 7. CONCLUZII ȘI DEZVOLTĂRI VIITOARE

7.1 Concluzii generale

Prin realizarea aplicației CodeScape s-a urmărit construirea unui mediu interactiv în care mai multe concepte introductive de programare să poată fi prezentate într-o formă mai accesibilă. Proiectul a demonstrat că un joc 3D poate fi folosit nu doar pentru divertisment, ci și ca suport pentru înțelegerea unor idei care, în mod obișnuit, sunt considerate abstracte.

Un rezultat important al proiectului este faptul că fiecare concept tratat a putut fi asociat cu o mecanică diferită, dar integrată într-un flux general coerent. Această diversitate a nivelurilor este utilă deoarece evită monotonia și arată că aceeași aplicație poate susține mai multe tipuri de activități educaționale.

Din punct de vedere tehnic, proiectul a permis utilizarea mai multor componente relevante din Unity și C#: control first-person, interacțiuni prin raycast, gestionarea prefabs-urilor, validarea unor condiții și controlul interfeței. Din punct de vedere didactic, a rezultat un instrument care poate fi folosit ca punct de plecare pentru explicații și demonstrații.

7.2 Gradul de atingere a obiectivelor

Obiectivele stabilite la începutul lucrării au fost atinse în mare măsură. A fost realizat un mediu 3D coerent, au fost implementate cinci niveluri tematice și a fost creat un sistem de progres bazat pe rezolvarea succesivă a provocărilor.

- obiectivul privind mediul 3D interactiv a fost îndeplinit prin implementarea controlului first-person și a unui spațiu explorabil;
- obiectivul privind introducerea conceptelor de programare a fost atins prin cele cinci niveluri tematice;
- obiectivul privind feedback-ul imediat a fost atins prin mesaje, efecte vizuale și semnale audio;
- obiectivul privind coerența aplicației a fost susținut de managerul de nivel și de organizarea pe camere succesive.

Există și elemente care pot fi dezvoltate în continuare, însă acestea țin mai degrabă de extinderea aplicației decât de lipsa unui nucleu funcțional. În forma actuală, proiectul își îndeplinește rolul asumat de prototip educațional complet și prezentabil.

7.3 Dezvoltări viitoare

O direcție firească de dezvoltare este extinderea numărului de niveluri. În aceeași structură pot fi introduse concepte suplimentare precum funcțiile, vectorii, variabilele sau chiar algoritmi elementari de sortare și căutare.

O altă direcție importantă este completarea aplicației cu un sistem de salvare a progresului și cu o evidență mai clară a performanței utilizatorului. Acest lucru ar transforma

jocul dintr-un prototip educațional punctual într-un instrument mai apropiat de o platformă de învățare.

În plus, ar putea fi utile și ajustări la nivel de prezentare: mai multe indicii vizuale, mai multe terminale explicative, posibilitatea de a selecta niveluri sau chiar integrarea unor diagrame și explicații suplimentare în meniuri. Toate aceste direcții sunt compatibile cu structura actuală a proiectului.

- adăugarea unor niveluri noi pentru alte concepte de programare;
- introducerea unui sistem de salvare a progresului;
- realizarea unui sistem de evaluare mai detaliat;
- adaptarea dificultății în funcție de nivelul utilizatorului;
- extinderea componentei informative prin terminale și capturi suplimentare.

BIBLIOGRAFIE

- [1] MIT Media Lab, "*About Scratch*", [Online]. Disponibil la: <https://scratch.mit.edu/help/about/>. Data accesării: 02.05.2026.
- [2] Lightbot, "*Hour of Code*", [Online]. Disponibil la: <https://lightbot.com/hour-of-code.html>. Data accesării: 02.05.2026.
- [3] CodeCombat, "*About CodeCombat*", [Online]. Disponibil la: <https://codecombat.com/about>. Data accesării: 03.05.2026.
- [4] Tomorrow Corporation, "*Human Resource Machine*", [Online]. Disponibil la: <https://tomorrowcorporation.com/humanresourcemachine>. Data accesării: 03.05.2026.
- [5] Unity Technologies, "*Unity Manual*", [Online]. Disponibil la: <https://docs.unity3d.com/2022.3/Documentation/Manual/index.html>. Data accesării: 08.05.2026.
- [6] Unity Technologies, "*Unity Scripting API*", [Online]. Disponibil la: <https://docs.unity3d.com/2022.3/Documentation/ScriptReference/>. Data accesării: 02.05.2026.
- [7] Microsoft, "*C# documentation*", [Online]. Disponibil la: <https://learn.microsoft.com/en-us/dotnet/csharp/>. Data accesării: 08.05.2026.
- [8] Unity Technologies, "*TextMesh Pro*", [Online]. Disponibil la: <https://docs.unity3d.com/Manual/com.unity.textmeshpro.html>. Data accesării: 10.05.2026.
- [9] Google Fonts, "*Jura*", [Online]. Disponibil la: <https://fonts.google.com/specimen/Jura>. Data accesării: 10.05.2026.
- [10] Unity Asset Store, "*Sci-Fi Styled Modular Pack*", [Online]. Disponibil la: <https://assetstore.unity.com/packages/3d/environments/sci-fi/sci-fi-styled-modular-pack-82913>. Data accesării: 13.05.2026.
- [11] Pixabay, "*Sound Effects*", [Online]. Disponibil la: <https://pixabay.com/sound-effects/>. Data accesării: 13.05.2026.